

xv6虚拟文件系统和硬件抽象层的设计与实现

赛题：2026年全国大学生计算机系统能力大赛-操作系统设计赛(全国)-OS功能挑战赛道 团队：tjuer 成员：陈权（VFS/ext2/FAT32）、杨子游（HAL/RISC-V/LoongArch） 日期：2026年8月16日

第一章 项目概述

1.1 项目背景

xv6-riscv是MIT开发的教学操作系统，运行于RISC-V平台，被广泛用于高校操作系统课程。其标准第5版存在两个核心局限：

- **单一文件系统**：仅支持自带的xv6文件系统（类Unix V6），无法与ext2、FAT32等主流文件系统交互
- **单一CPU架构**：内核代码与RISC-V紧耦合，移植到其他架构需大量重写

这使得xv6在教学中难以展示"Linux是如何同时支持ext4/xfs/btrfs的"以及"操作系统如何跨平台运行"等关键概念。

本项目为xv6-riscv第5版扩充**虚拟文件系统（VFS）**和**硬件抽象层（HAL）**，支持xv6fs、ext2、FAT32三种文件系统以及RISC-V、LoongArch两种CPU架构。实现保留xv6教学内核的代码规模和模块结构。

1.2 项目目标

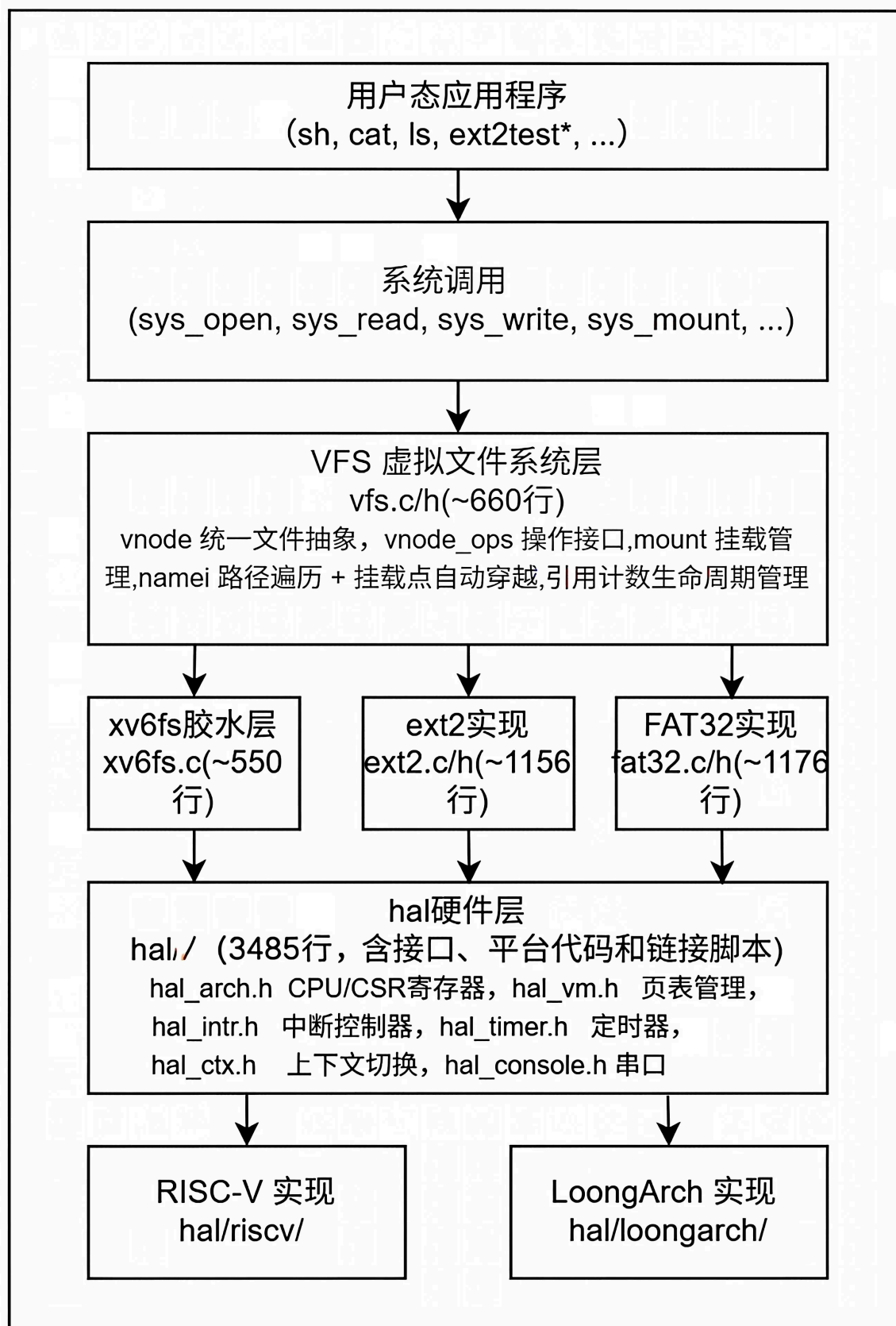
编号	目标	说明
G1	同时支持3种文件系统	xv6fs（原生）、ext2（Linux早期标准）、FAT32（U盘通用）
G2	同时支持2种CPU架构	RISC-V（已验证）、LoongArch（国产自主架构）
G3	通过所有测例	xv6原生usertests + 赛方测试用例 + 自定义综合测试
G4	代码尽量少	复用原xv6成熟路径，删除重复包装和死代码，把新增机制限制在必要的平台差异与文件系统能力内
G5	运行稳定	无panic、无内存泄漏、并发安全

1.3 文档结构导览

章节	内容	读者对象
第一章	项目概述（本文）	所有读者
第二章	VFS设计详解	关注虚拟文件系统核心算法的读者
第三章	ext2适配详解	关注具体文件系统接入过程的读者
第四章	xv6fs胶水层	关注原生文件系统适配的读者
第五章	HAL设计概述	关注抽象边界和公共接口的读者

章节	内容	读者对象
第六章	RISC-V平台适配	关注RISC-V启动、VM和设备驱动的读者
第七章	LoongArch平台适配	关注LoongArch移植机制的读者
第八章	FAT32适配	关注FAT32实现的读者
第九章	测试与验证	评审专家、需要复现结果的读者
第十章	设计精巧度分析	评审专家
第十一章	挑战与展望	所有读者

1.4 总体架构



总体结构：VFS层通过统一vnode抽象隔离上层系统调用与具体文件系统；HAL层通过统一接口头文件隔离内核核心逻辑与CPU架构。文件系统扩展依赖VFS接口，CPU移植依赖HAL接口，两条扩展路径保持正交。

1.5 团队分工

成员	负责模块	具体内容
陈权	VFS + 多文件系统	VFS核心设计、xv6fs胶水层、ext2完整实现、FAT32实现、综合测试
杨子游	HAL + 多架构	HAL接口设计、RISC-V平台实现、LoongArch平台移植

1.6 功能模块一览

功能模块	说明
VFS核心层	mount/umount、namei路径遍历、挂载点穿越、"."跨FS返回、readdir统一ABI (vdirent/VDIRSIZ)
xv6fs胶水层	原生inode零修改包装为vnode
ext2文件系统	完整vnode_ops（13个函数），支持文件/目录/硬链接/设备节点，长文件名（≤255）
FAT32文件系统	11/13 个 vnode_ops（FAT32 无 inode，不实现 link/mknod），8.3短名 + VFAT LFN长文件名（≤255），O_APPEND/O_TRUNC，双平台
RISC-V HAL	启动、Sv39页表、PLIC/SSTC中断定时器、中断驱动UART、1~3盘virtio磁盘
LoongArch HAL	启动、四级页表/TLB软重填、EIOINTC/IOCSR中断驱动UART、10Hz定时器、三盘RAM disk

1.7 当前代码规模

类别	文件	行数
VFS核心	kernel/vfs.c + kernel/vfs.h	581 + 114 = 695行
ext2实现	kernel/ext2.c + kernel/ext2.h	1070 + 128 = 1198行
xv6fs胶水	kernel/xv6fs.c	497行
FAT32	kernel/fat32.c + kernel/fat32.h	1056 + 91 = 1147行
HAL接口层	hal/hal.h + hal/hal_*.h（8个接口头文件）	234行
RISC-V HAL	hal/riscv/（13个文件，含链接脚本）	1608行
LoongArch HAL	hal/loongarch/（14个文件，含链接脚本）	1643行
上层适配	kernel/sysfile.c (386行) + kernel/file.c (163行)	549行
用户测试	user/ext2test.c + user/fat32test.c + user/test1.c + user/test2.c	853 + 546 + 64 + 81 = 1544行
统计说明	表中数据采用当前文件规模口径；HAL共3485行，包含从原xv6迁入的RISC-V平台代码，VFS适配复用了原有内核路径	

第二章 VFS虚拟文件系统设计

2.1 设计动机与目标

2.1.1 原xv6的局限

xv6-riscv-rev5 的文件系统代码 (`kernel/fs.c`、`kernel/log.c`) 存在三个固有限制：

- 1. **硬编码文件系统类型**：所有文件操作直接操作 `struct inode`，inode 管理 (`ialloc`、`iget`、`iput`)、目录逻辑 (`dirlookup`、`dirlink`)、块映射 (`bmap`) 全部假设底层是 xv6 原生 FS 格式。要支持 ext2，必须重写整个文件操作栈。
- 2. **无挂载点概念**：只有一个根文件系统，无法像 Linux 那样在 `/mnt` 上挂载第二块磁盘。根 inode 从超级块直接读取，不存在 "不同设备上不同文件系统" 的抽象。
- 3. **路径遍历与 FS 实现耦合**：`namei` / `nameiparent` 内联调用 `dirlookup` (xv6fs 特有)，无法遍历 ext2 目录项。

2.1.2 VFS 的设计目标

目标	具体含义
统一接口	所有文件操作通过 <code>vnode_ops</code> 虚拟表调用，系统调用不感知底层 FS 类型
多 FS 支持	可同时挂载 xv6fs (<code>root</code>) 和 ext2 (<code>/mnt</code>)，路径遍历自动在挂载点间穿越
最小侵入	<code>kernel/fs.c</code> 和 <code>kernel/log.c</code> 一行不改，xv6fs 作为独立胶水层包装原 inode
代码精简	<code>vfs.c</code> + <code>vfs.h</code> 合计约 695 行，符合教学操作系统定位
依赖倒转	VFS 层不依赖具体 FS 实现——FS 通过 <code>vfs_register</code> 注册自身，VFS 只认 <code>vnode_ops</code> 接口

2.1.3 设计原则

- **不随复杂度膨胀**：宁可一个 `vnode` 统一抽象，不要 `dentry` / `inode` / `address_space` 多层解体
- **引用计数驱动**：所有 `vnode` 生命周期由 `ref` 精确管理，`vput` 触发延迟销毁
- **错误路径可恢复**：`mount` 失败时完整回滚，不泄漏内存或锁

2.2 整体架构

VFS 层嵌于系统调用与具体文件系统之间，对上提供 28 个公共 API（其中 23 个以 `vfs_` 命名，全部在 `kernel/vfs.h` 中声明），对下通过两套虚拟表 `vnode_ops`（文件级）和 `vfs_ops`（挂载级）驱动具体实现。横向通过全局 `mounttable[NMOUNT]` 管理最多 8 个挂载点，`namei` 路径遍历在挂载点间自动穿越。

![屏幕截图 2026-08-15 160410](决赛实验报告.assets/屏幕截图 2026-08-15 160410.png)

关键交互路径：

- **上对 `sysfile.c`**：所有系统调用通过 `vfs_open` / `vfs_read` / `vfs_write` / `vfs_close` / `vfs_mount` / `vfs_umount` 等薄包装进入 VFS
- **下对具体 FS**：VFS 不持有 FS 实现的任何头文件引用——仅通过 `fstypes[]` 注册表在 `mount` 时调用 `mountfn(dev)`，之后通过 `vnode_ops` 回调
- **横向挂载管理**：`namei` 中每步 `lookup` 后调用 `cross_mount` 检查是否为挂载点，自动穿越到下级文件系统

2.3 核心数据结构

2.3.1 `vnode` — 统一文件抽象

```
struct vnode {
    uint type;           // V_FILE(1), V_DIR(2), V_DEV(3)
    uint dev;           // 所属设备的设备号, 从 mp->dev 继承
    int major;          // V_DEV 的主设备号
    int minor;          // V_DEV 的次设备号
    struct vnode_ops *ops; // 操作虚拟表, 由所属 FS 在创建时填充
    struct mount *mp;    // 所属挂载点; ref==0 时置 NULL
    int ref;             // 引用计数: vget 递增, vput 递减到 0 触发销毁
    struct sleeplock lock; // 保护 vnode 读写操作的睡眠锁
    uint inum;           // FS 内部 inode 号, 仅在所属 mp 内有意义
    uint ikey;           // 二级身份键 (FAT32: 目录项偏移 dent_off; 其他 FS 为 0)
    uint size;           // 缓存的文件大小, 由 FS 后端维护 (O_APPEND 等)
    void *priv;          // FS 私有数据指针 (如 xv6fs 的 struct inode*)
};
```

字段详解：

字段	作用	生命周期
type	区分文件/目录/设备节点	alloc_vnode 置 0, FS 的 create/lookup 设置为 V_FILE/V_DIR/V_DEV
dev	用于 mountpoint 匹配和 I/O 路由	从mp->dev 继承, vnode 销毁时不变
inum	配合 dev 在系统内唯一标识一个文件	FS 的 create/lookup 填入
ref	防止正在使用的 vnode 被回收	alloc_vnode 置 1, vget / vput 原子操作
ops	解耦 VFS 与具体 FS: VFS 只调 ops, 不关心实现	FS 的 create/lookup 填充, vput 销毁前置 NULL
mp	反向指针, 用于vfs_find_mountpoint 和 vfs_link 跨FS检查	由vfs_mount 或 FS create 设置
priv	避免 vnode 膨胀——FS 自行分配私有结构挂载于此	FS create/lookup 分配, ops->destroy 释放
size	缓存文件大小, file_write 的 O_APPEND 依赖此字段	FS 的 read/write/truncate 维护
lock	串行化对同一 vnode 的并发读写 (如 file_read/file_write)	初始化时 initsleeplock, 销毁时隐式释放
ikey	与inum 组合构成 vnode 去重的二级身份键: FAT32 因无 inode 号, 用「父目录首簇 + 目录项偏移」标识文件, ikey 存目录项偏移; 其余 FS 为 0	FAT32 的fvalloc 在创建时填充, vfs_iget 用它匹配活跃 vnode

2.3.2 vnode_ops — 文件操作接口（13 个函数指针）

vnode_ops 是 VFS 设计的核心抽象，每个具体文件系统实现这 13 个函数，VFS 通过 VOP_* 宏间接调用。

序号	函数指针	签名	功能
1	lookup	int(vnode*, char*, vnode**)	在目录中查找 name，返回目标 vnode（ref 已持有）。失败返回非0
2	mknod	int(vnode*, char*, int, int)	在目录中创建设备节点（major, minor）
3	read	int(vnode*, uint64, int, uint)	从 off 偏移读 n 字节到用户地址 buf；返回实际读取字节数
4	write	int(vnode*, uint64, int, uint)	将用户地址 buf 的 n 字节写入 off 偏移；返回实际写入字节数
5	stat	int(vnode*, uint64)	填充struct stat 到用户地址 addr
6	readdir	int(vnode*, uint64, uint)	从 off 偏移读一个目录项到用户地址 buf；返回下一偏移
7	create	int(vnode*, char*, short, vnode**)	在 dir 中创建 type 类型的 inode；失败返回非0
8	unlink	int(vnode*, char*)	从 dir 移除 name；若 nlink==0 则释放 inode（可能延迟到 vput）
9	mkdir	int(vnode*, char*)	在 dir 中创建子目录（含 "." 和 ".."）
10	link	int(vnode*, char*, vnode*)	将 old vnode 硬链接到 dir 下名为 name
11	read_kernel	int(vnode*, uint64, int, uint)	内核态读——直接写入内核地址 buf，避免 copyout 开销（exec 用）
12	truncate	int(vnode*)	释放所有数据块，设置 size 为 0；仅 V_FILE
13	destroy	void(void*)	释放vnode->priv 指向的 FS 私有数据，在 vput 的锁外延迟调用

VFS 层通过 VOP_* 宏简化调用：

```
#define VOP_LOOKUP(v,n,r)      (v)->ops->lookup(v,n,r)
#define VOP_READ(v,a,n,o)     (v)->ops->read(v,a,n,o)
#define VOP_WRITE(v,a,n,o)    (v)->ops->write(v,a,n,o)
// ... 共 13 个宏
```

接口设计原则：

- read / write 使用 uint64 作为用户地址（兼容 64 位指针，内核态通过 copyin / copyout 访问）
- read_kernel 以直接内存访问替代 readi，消除 exec 加载 ELF 时的 double-copy 开销
- 所有创建操作（create / mkdir / mknod）在持有 vn_lock 下调用，保证目录完整性

2.3.3 vfs_ops — 挂载级操作接口

```
struct vfs_ops {
    struct vnode* (*root)(struct mount*);    // 返回本 FS 的根 vnode (ref 已持有)
    void (*unmount)(struct mount*);          // 回写超块 + 释放 FS 私有挂载数据
};
```

vfs_ops 只定义了两个函数指针，比 vnode_ops 简单得多。`root` 在 mount 完成时调用一次，`unmount` 在 umount 时调用。

2.3.4 mount 与 mounttable

```
struct mount {
    int valid;                // 1 表示已成功注册到 mounttable (原子状态标记)
    struct vfs_ops *ops;      // root() + unmount()
    struct vnode *root;       // 本 FS 的根 vnode (ref 永久持有)
    struct vnode *mountpoint; // 挂载点 vnode (父 FS 的目录 vnode); 根挂载为 NULL
    uint dev;                 // 本挂载涉及的所有 I/O 的目标设备号
    char path[128];           // 挂载路径字符串 (调试用; 实际遍历用 mountpoint 指针)
    void *priv;               // FS 私有挂载数据 (如 ext2 的超块缓冲区)
};
```

挂载表是全局数组：

```
struct mount *mounttable[NMOUNT]; // NMOUNT=8
int nmount;                       // 当前活跃挂载数
```

示例布局 (同时挂载 xv6fs 和 ext2)：

```
mounttable[0] = { valid=1, dev=0, root=vA, mountpoint=NULL, path="/" }    // xv6fs
根挂载
mounttable[1] = { valid=1, dev=1, root=vB, mountpoint=vC, path="/mnt" }    // ext2
挂载在 /mnt
```

其中 vA 是 xv6fs 根 inode 的 vnode，vB 是 ext2 根 inode 的 vnode，vC 是 xv6fs 中 `/mnt` 目录的 vnode。

2.4 路径遍历算法

2.4.1 vfs_namei 完整流程

`vfs_namei` 是 VFS 的——它将路径字符串解析为目标 vnode，并在每次遍历目录项后自动穿越挂载点。函数约 45 行，逻辑如下：

```
vfs_namei(path):
    1. path_start(path, &rest)    // '/' → vget(mounttable[0]->root)
                                   // 否则 → vget(p->cwd)
```



```

2. while (rest = skipelem(rest, name)) != 0:
    vn_lock(vp)
    if vp->type != V_DIR → 失败, 返回 NULL

    // ".." 跨 FS 特殊处理 (见 2.4.3)
    if name==".." && vp 是其 FS 的 root:
        mpoint = vget(vp->mp->mountpoint)
        vn_unlock(vp); vput(vp)
        vn_lock(mpoint); mpoint->ops->lookup(mpoint, "..", &next)
        vn_unlock(mpoint); vput(mpoint)
        vp = next
        continue

    // 正常 lookup
    vp->ops->lookup(vp, name, &next) // FS 后端查找目录项, 返回 vnode
    vn_unlock(vp); vput(vp)

    vp = cross_mount(next) // 检查下一跳是否为挂载点, 若是则穿越
3. return vp // ref 已持有, 调用方负责 vput

```

关键细节:

- `path_start` 返回的 vnode 已持有 ref (通过 `vget`), 确保遍历期间不被回收
- 每步 `lookup` 返回的 vnode 的 ref 由 FS 后端在 `lookup` 内部持有 (对应 `alloc_vnode` 语义)
- 逐级 `vput(vp)` 释放前一级 vnode, 保证引用计数精确平衡
- `cross_mount` 消费传入的 ref, 返回新 ref, 无需额外引用计数操作

2.4.2 挂载点穿越 (cross_mount)

```

static struct vnode*
cross_mount(struct vnode *vp)
{
    struct mount *submp;
    submp = vfs_lookup_mount(vp);
    if(submp){
        vput(vp);
        return vget(submp->root); // 返回挂载在 vp 上的 FS 的根 vnode
    }
    return vp; // 非挂载点, 原样返回
}

```

`vfs_lookup_mount` 遍历 `mounttable[1..nmount-1]` (跳过根挂载), 用 `dev+inum` 匹配:

```

if(mounttable[i]->mountpoint &&
    mounttable[i]->mountpoint->dev == vp->dev &&
    mounttable[i]->mountpoint->inum == vp->inum)
    return mounttable[i];

```

挂载点采用dev+inum判定

不同 FS 对同一个 inode 调用 `vget` (或 `xv6fs` 的 `iget`) 可能返回**不同地址的 vnode 对象** (`vnode_table` 中的不同槽位), 但 `dev` 和 `inum` 在系统内是全局唯一的 (`dev` 标识设备, `inum` 是 FS 内 inode 号)。用 `dev+inum` 匹配保证了正确性, 避免了因缓存复用导致的指针失效问题。

2.4.3 ".." 跨文件系统处理

处理场景: `ext2` 挂载在 `xv6fs` 的 `/mnt` 上。用户在 `ext2` 根目录执行 `cd ..` 时, 路径解析返回 `xv6fs` 中的 `/mnt` 目录。

实现逻辑 (嵌入在 `namei` 的循环中):

```
if(strncmp(name, "..", 15) == 0 &&
    vp->mp && vp->mp->mountpoint &&
    vp->dev == vp->mp->root->dev &&
    vp->inum == vp->mp->root->inum)
{
    struct vnode *mpoint = vget(vp->mp->mountpoint);
    vn_unlock(vp);
    vput(vp);
    vn_lock(mpoint);
    mpoint->ops->lookup(mpoint, "..", &next);
    vn_unlock(mpoint);
    vput(mpoint);
    vp = next;
    continue;
}
```

图解:

```
xv6fs:
    / → /mnt (vnode C) — ".." → / (vnode A)
      |
ext2:  | 挂载于此
    /ext2root (vnode B) — ".." → ?
```

处理流程:

- `cd ..` 在 `ext2` 根目录执行
- 检测 `vB.dev==mp->root->dev && vB.inum==mp->root->inum`
- 是 `root` → 穿越到 `mp->mountpoint` (`vC`, `xv6fs` 的 `/mnt`)
- 在 `xv6fs` 中 `lookup(mpoint, "..")` → 返回 `/` (`vA`)
- 最终结果: `vA` (`xv6fs` 根目录) ✓

与 Linux 的对比: Linux 通过 `dentry` 的 `d_parent` 指针和 `follow_up` 实现相同语义。本 `vnode` 实现比 Linux 简化——不维护双向父子链接, 仅在 `".."` 查询时动态穿越。

2.4.4 vfs_nameparent

`vfs_nameparent` 是 `vfs_namei` 的变体, 用于 `O_CREATE` 路径:

```
vfs_nameiparent("/mnt/newfile", name):
→ 逐级遍历到 /mnt 的 vnode
→ 保存最终分量 "newfile" 到 name
→ 返回 /mnt 的 vnode (ref 持有)
```

调用方拿到父目录 vnode 后，在其上调用 `VOP_CREATE(dir, name, ...)` 创建新文件。这避免了先 `namei` 再解出父目录的双重遍历。

2.5 文件系统注册与挂载

2.5.1 vfs_register — 注册文件系统类型

VFS 维护一个静态注册表：

```
struct {
    struct mount* (*mountfn)(uint); // mount 入口函数：接收 dev，返回 mount 结构
    char name[16]; // FS 类型名称 ("xv6fs", "ext2")
} fstypes[8];
int nfstype; // 已注册数量，上限 8
```

注册接口如下：

```
void vfs_register(struct mount* (*fn)(uint), char *name)
{
    if(nfstype >= 8) panic("vfs_register: too many fstypes");
    fstypes[nfstype].mountfn = fn;
    safestrcpy(fstypes[nfstype].name, name, 16);
    nfstype++;
}
```

`vfs_init` 在启动时调用三次注册：

```
vfs_register(xv6fs_mount, "xv6fs");
vfs_register(ext2_mount, "ext2");
vfs_register(fat32_mount, "fat32");
```

添加新 FS 只需一行 `vfs_register(...)`，无需修改任何 VFS 内部逻辑——FAT32 的 LoongArch 平台支持正是决赛阶段复用该注册机制快速接入的（其 RISC-V 基础后端于初赛即已注册）。

2.5.2 vfs_mount — 挂载完整流程

`vfs_mount` 实现 7 个步骤，每步失败均有对应回滚：

```

vfs_mount(path, dev, fstype):
1. 重复挂载检查
   for i in 0..nmount-1:
       if mounttable[i]->path == path → return 0 (同名路径已挂载)

2. 查找 FS 类型
   for i in 0..nfstype-1:
       if fstypes[i].name == fstype:
           mp = fstypes[i].mountfn(dev) // 调用 xv6fs_mount / ext2_mount
   if mp == 0 → return 0

3. 解析挂载点 vnode
   if path == "/":
       mp->mountpoint = NULL
   else:
       mpoint = vfs_namei(path)
       if mpoint==NULL || mpoint->type!=V_DIR:
           → 回滚: vput(mpoint); mp->ops->unmount(mp); kfree(mp); return 0

4. 重复挂载检查 (同一 vnode)
   for i in 1..nmount-1: // 跳过 mounttable[0]
       if mounttable[i]->mountpoint->dev==mpoint->dev &&
           mounttable[i]->mountpoint->inum==mpoint->inum:
           → 回滚: vput(mpoint); mp->ops->unmount(mp); kfree(mp); return 0

5. 填充 mount 结构
   mp->mountpoint = mpoint
   safestrcpy(mp->path, path, 128)
   mp->dev = dev

6. 容量检查
   if nmount >= NMOUNT:
       → 回滚: vput(mpoint); mp->ops->unmount(mp); kfree(mp); return 0

7. 注册到 mounttable
   mounttable[nmount] = mp
   nmount++
   mp->valid = 1
   return mp

```

错误恢复路径的关键：

- 步骤 3~6 的失败回滚都执行 `mp->ops->unmount(mp) + kfree(mp)`——因为 `mountfn` 已在步骤 2 中分配了 mount 结构和 FS 私有数据
- `vput(mpoint)` 释放步骤 3 获取的挂载点 vnode
- 不回滚 `fstypes[]`——FS 类型注册是全局的，一次注册永久有效

2.5.3 vfs_umount — 卸载流程

```

int vfs_umount(char *path)
{

```

```

// 1. 在 mounttable 中按 path 字符串查找
for(idx=0; idx<nmount; idx++)
    if(strncmp(mounttable[idx]->path, path, 128)==0)
        mp = mounttable[idx];

// 2. 安全检查
if(mp==0 || !mp->valid) return -1;
if(idx == 0) return -1; // 根挂载不可卸载

// 3. 调用 FS 的 unmount 回调（回写超块、释放私有数据）
if(mp->ops && mp->ops->unmount)
    mp->ops->unmount(mp);

mp->valid = 0;

// 4. mounttable 前移压缩（O(n) 删除，保留顺序）
for(i=idx; i<nmount-1; i++)
    mounttable[i] = mounttable[i+1];
mounttable[nmount-1] = 0;
nmount--;

// 5. 释放挂载点 vnode
if(mp->mountpoint) vput(mp->mountpoint);

// 6. 释放 mount 结构体
kfree((void*)mp);
return 0;
}

```

注意事项:

- 查找键使用 `path` 字符串；`dev` 无法区分多个设备上的同名路径
- 卸载不以 root vnode 的 ref 计数为条件——xv6 中假定调用 `umount` 时没有打开此 FS 上的文件（POSIX `umount` 的 `EBUSY` 检查省略）
- `mounttable` 删除通过数组前移实现， $O(n)$ 但 `NMOUNT=8` 时微不足道

2.6 vnode 生命周期管理

2.6.1 vnode 存储

vnode 使用 **静态数组 + 线性扫描** 分配。当前实现未引入 Linux 式 slab 缓存：

```

struct vnode vnode_table[NVNODE]; // NVNODE=200, 约 200×80B ≈ 16KB
struct spinlock vnode_lock;        // 保护 ref 字段和分配状态

```

`alloc_vnode` 遍历 `vnode_table` 查找 `ref==0` 的空闲槽位，置 `ref=1` 并清零其他字段：

```

struct vnode* alloc_vnode(void)
{
    acquire(&vnode_lock);

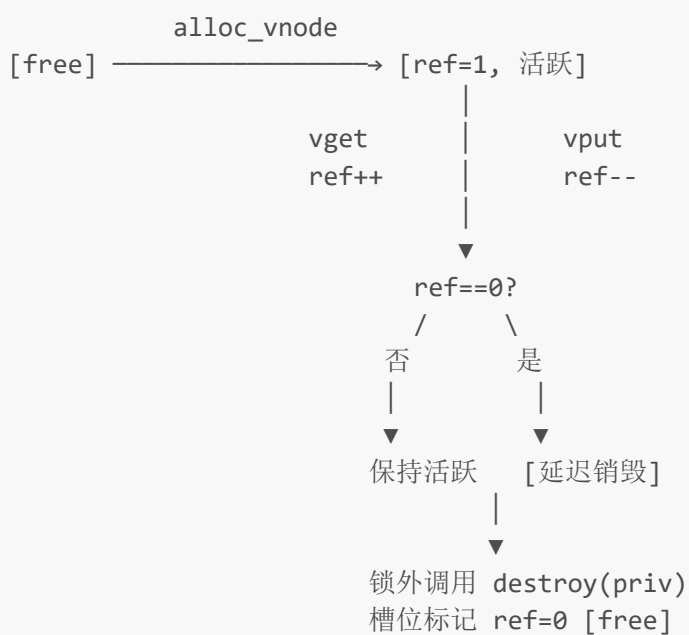
```

```

for(i = 0; i < NVNODE; i++){
    if(vnode_table[i].ref == 0){
        vnode_table[i].ref = 1;
        vnode_table[i].type = 0;
        vnode_table[i].ops = 0;
        vnode_table[i].mp = 0;
        vnode_table[i].priv = 0;
        vnode_table[i].inum = 0;
        vnode_table[i].major = 0;
        vnode_table[i].minor = 0;
        release(&vnode_lock);
        return &vnode_table[i];
    }
}
release(&vnode_lock);
return 0; // 返回 NULL 表示 vnode 池耗尽
}

```

2.6.2 引用计数状态机



- **vget:** `acquire(&vnode_lock); vp->ref++; release(&vnode_lock);` — 原子递增，调用者确保 vp 当前合法
- **vput:** `acquire(&vnode_lock); vp->ref--; if(vp->ref==0) need_free=1; release(&vnode_lock);` — ref 降到 0 时不立即销毁，而是在释放自旋锁之后异步调用 `destroy`

2.6.3 延迟销毁机制

```

void vput(struct vnode *vp)
{
    void *priv;
    void (*destroy)(void*);
}

```

```

int need_free = 0;

acquire(&vnode_lock);
vp->ref--;
if(vp->ref == 0){
    need_free = 1;
    priv = vp->priv;                // 快照私有数据
    destroy = (vp->ops) ? vp->ops->destroy : 0; // 快照 destroy 函数
    vp->priv = 0;                  // 清空槽位字段
    vp->ops = 0;
    vp->mp = 0;
    vp->type = 0;
}
release(&vnode_lock);            // 先释放锁

if(need_free && destroy)
    destroy(priv);                // 再调用 destroy
}

```

destroy延迟到锁外执行

`destroy(priv)` 最终会调用具体 FS 的清理函数（如 xv6fs 的 `iput` → `iunlockput`），后者可能涉及块分配/释放、日志写入、缓冲区操作——这些操作内部会获取其他锁。如果 `vput` 持有 `vnode_lock` 自旋锁时调用 `destroy`，会导致：

1. **死锁**：`destroy` → `begin_op` → `log.lock`，同时另一线程持有 `log.lock` 等待 `vnode_lock`
2. **自旋锁下睡眠**：块 I/O 可能触发 `sleep`，而在自旋锁下睡眠是违规的

延迟销毁通过在锁外调用 `destroy` 完美解决了这两个问题。这也是本设计中引用计数管理的核心诀窍。

2.6.4 活跃 vnode 去重 (`vfs_iget`)

`vfs_iget` 镜像 xv6 `iget` 的“同一性保证”：同一文件在内存中只有一个规范 `vnode`。它在持有 `vnode_lock` 时按 `(mp, inum, ikey)` 线性扫描 `vnode_table`，若命中活跃 `vnode` 则 `ref++` 并返回复用；未命中返回 0，由调用方分配新槽。

```

struct vnode* vfs_iget(struct mount *mp, uint inum, uint ikey)
{
    acquire(&vnode_lock);
    for(v = vnode_table; v < vnode_table + NVNODE; v++){
        if(v->ref > 0 && v->mp == mp && v->inum == inum && v->ikey == ikey){
            v->ref++;
            release(&vnode_lock);
            return v;
        }
    }
    release(&vnode_lock);
    return 0;
}

```

没有去重时，同一文件被两次 `open` 会产生两个独立 `vnode`，各自缓存 `file_size/first_clus` 等私有快照：并发写入时一个 `vnode` 用陈旧快照回写磁盘，会覆盖另一个 `vnode` 刚更新的数据（文件大小/首簇回退、数据串扰）。接入 `vfs_iget` 后所有 `fd` 共享同一 `vnode` 与同一把 `vn_lock`，并发读写天然串行化。

各 FS 的接入方式：

- **ext2**： `ext2_iget` 先调 `vfs_iget(mp->mnt, inum, 0)`（ext2 有 inode 号，无需二级键）。
- **FAT32**： `fvalloc` 先调 `vfs_iget(mp, pdir, deoff)`。FAT32 无 inode 号且空文件 `first_clus==0`，不能以首簇为键；身份键采用**目录项位置**（父目录首簇 `pdir`，目录项偏移 `deoff`），这才是磁盘上唯一稳定的文件标识。

2.7 跨文件系统操作限制

VFS 对跨挂载点的操作实施限制，保证 FS 内部数据结构的完整性。

2.7.1 硬链接限制（`vfs_link`）

```
int vfs_link(struct vnode *old, struct vnode *newdir, char *name)
{
    if(old->mp != newdir->mp) return -1; // 跨 FS 硬链接拒绝
    // ...
    return VOP_LINK(newdir, name, old);
}
```

硬链接的本质是同一个 FS 内部的 inode 别名——两个目录项指向同一个 inode 号。跨 FS 的硬链接无意义（ext2 的 inode 对 xv6fs 不可见）。Linux 对此返回 `EXDEV` 错误。

2.7.2 `rename` 的简化策略

xv6 VFS 不实现 `vfs_rename`——采用 `link + unlink` 的简化语义。这一方面是因为 xv6 原生的 `sys_rename` 不存在，另一方面 `rename` 的跨 FS 原子迁移非常复杂（涉及源 FS 的 orphan 和目的 FS 的数据拷贝），对教学操作系统而言代价过高。

2.7.3 其他跨 FS 操作

操作	跨 FS 行为	理由
<code>vfs_read / vfs_write</code>	允许（自动跟随 <code>vnode->ops</code> ）	只操作已打开的文件
<code>vfs_open</code>	允许（ <code>namei</code> 自动穿越挂载点）	路径遍历透明
<code>vfs_create</code>	不允许（必须在同一 <code>dir</code> 内）	<code>create</code> 在单个 <code>dir</code> <code>vnode</code> 上操作
<code>vfs_unlink</code>	不允许（删除在 <code>dir</code> 内完成）	同上

2.8 设计决策记录

决策	原因
<code>cwd</code> 使用 <code>vnode</code> 指针	目录 <code>rename</code> 后 <code>cwd</code> 仍指向同一 <code>vnode</code> ，路径字符串可能失效

决策	原因
priv 使用void指针	VFS与具体FS保持解耦，新增FS不修改vnode结构
NVNODE=200	原 xv6 NINODE=50；vnode 需更多：open file (NFILE=128) + cwd + namei 中间节点 + 跨 FS 挂载点
mountpoint 用 dev+inum 匹配	不同 iget 可能返回不同地址的 vnode 对象；dev+inum 在系统内全局唯一
vput 延迟销毁（锁外 destroy）	避免 destroy 中的块分配/日志操作导致死锁或自旋锁下睡眠
线性扫描 vnode 缓存（非哈希）	代码精简；NVNODE=200 时 O(N) 扫描开销可接受（~0.2μs）
namei 不使用 dentry 缓存	避免缓存失效的复杂性；路径遍历频率在 xv6 中很低
mounttable 固定数组	NMOUNT=8 足够（根 + 7 个额外 FS）；动态链表增加复杂而无实际收益
一对一路径绑定	一个目录只能挂载一个文件系统，同一设备只提供一份 FS 实例（不嵌套、不重复挂载）。mount 表以 mountpoint→root 单射、dev+inum 全局唯一标识 FS 根，使路径遍历与挂载点穿越保持约 45 行的精简实现。代码层通过路径查重与挂载点 vnode（dev+inum）查重施加该约束
umount 不检查 busy	xv6 中调用 umount 前应确保无打开文件（简化实现，未来可加 EBUSY 检查）

2.9 涉及文件与代码量

文件	行数	角色
kernel/vfs.h	114行	vnode / vnode_ops / mount 类型定义, VOP_* 宏, API 声明
kernel/vfs.c	581行	namei 路径遍历, mount/umount, vnode 生命周期, vfs_iget 去重, 约27个 API
kernel/file.h	25行	struct file 中 f->ip 改为 struct vnode*
kernel/file.c	163行	file_read/file_write 调用 vfs_read/vfs_write；fileclose 调用 vput
kernel/sysfile.c	386行	所有 sys_* 调用 VFS API（sys_open, sys_mount, sys_umount 等）
kernel/proc.h	89行	cwd 字段改为 struct vnode*
kernel/exec.c	171行	exec 调用 vfs_namei + vfs_read_kernel 加载 ELF
kernel/fcntl.h	6行	O_CREATE / O_APPEND / O_TRUNC 等标志位定义

核心文件: `vfs.h` + `vfs.c`（695行，包含全部VFS架构和逻辑） **关联文件:** 6个上层文件（`sysfile.c`, `file.c`, `file.h`, `exec.c`, `proc.h`, `fcntl.h`）

第三章 ext2文件系统适配

3.1 ext2简介

ext2（Second Extended Filesystem）是Linux早期标准的文件系统，由Rémy Card于1993年设计。它采用经典的Unix文件系统布局，包括超级块、块组描述符、inode位图、块位图、inode表和data blocks。本项目使用这些结构展示第二种磁盘文件系统的接入过程。

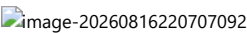
本实现以约**1000行C代码**完整实现了ext2的基础读写能力（含文件/目录/设备节点的创建、读写、删除、硬链接和时间戳维护），并通过vnode_ops接口无缝接入VFS层，使得上层应用程序可以用与xv6fs完全相同的系统调用操作ext2文件系统。

3.2 适配策略

策略	说明
仅支持1024字节块	<code>s_log_block_size == 0</code> ，与xv6 BSIZE对齐，直接复用buf-cache层（ <code>bread/bwrite</code> ）
独立设备（决赛版）	ext2 使用第二块独立磁盘 <code>ext2.img</code> （ <code>dev=2</code> ），挂载点 <code>/mnt</code> ；超级块直接读取块1、块组描述符读取块2，无任何偏移
128字节基础inode	支持rev=0的128字节inode，同时通过 <code>s_inode_size</code> 动态检测256字节inode
直接块 + 单间接块	<code>i_block[0..11] + i_block[12]</code> 单向间接，最大文件268KB
不支持	扩展属性、ACL、日志、extent格式、双重/三重间接块、HTREE索引目录

初赛阶段曾采用"xv6fs+ext2 共享 fs.img 的 FSSIZE 偏移方案"（单磁盘双分区），决赛阶段改为三文件系统三设备——对应布局（fs=dev=1、ext2=dev=2、fat32=dev=3），消除了偏移计算的复杂度，也使三 FS 在 RISC-V/LoongArch 上的设备语义完全一致。

独立设备布局图：



RISC-V：三个独立 virtio 磁盘（`-drive`）；LoongArch：三个独立 loader RAM disk（`0x09000000/0x0a000000/0x0b000000`）。

3.3 数据流设计

用户态 `read()` 到ext2磁盘操作的完整链路：

```
用户程序：read(fd, buf, 100)
|
▼
sys_read() [sysfile.c]
→ fileread(f, addr, n) [file.c]
→ vn_lock(vp) // 获取vnode睡眠锁，串行化并发I/O
→ VOP_READ(vp, addr, n, off) // 宏展开 → vp->ops->read(vp, addr, n, off)
→ ext2_read(vp, buf, n, off) [ext2.c]
  | 检查 off >= i_size → 返回0 (EOF)
  | 调整 n 不超过文件末尾
  | while(total < n):
  |   | ext2_bmap(priv, off/BSIZE, 0) // 逻辑块号→物理块号（不分配，alloc=0)
```

```
| | | | lbn < 12 → 直接块，读取 ino->i_block[lbn]
| | | | └ lbn 12..267 → 读间接块 → 查表 → 返回物理块号
| | | └ blk == 0 → 稀疏空洞，填充零到用户buf
| | └ bread(mp->dev, blk) // 通过buf-cache读取磁盘块
| └ copyout(p->pagetable, buf+total, bp->data+boff, len)
└ ext2_update_time(priv, 1, 0) // 更新atime
→ vn_unlock(vp)
→ 返回实际读取字节数
```

关键设计点:

- `ext2_bmap` 两次出镜：read时 `alloc=0`（只读不分配），write时 `alloc=1`（按需分配）
- `bread/bwrite` 复用xv6的块缓存，无需实现独立的磁盘I/O层
- `dev=2`本身就是完整ext2镜像，ext2块号可直接传给`bread()`，不再维护组合镜像偏移
- `vnode`锁在 `fileread/filewrite` 中获取，保证ext2层内的操作是串行化的

3.4 核心数据结构

3.4.1 ext2 mount priv — 挂载级私有数据

每次 `ext2 mount(dev)` 在堆上分配一个 `ext2 mount priv` 结构体, 挂在 `mount->priv` 上:

```
struct ext2_mount_priv {
    struct ext2_superblock sb;           // 内存中的超块副本（从块1读取）
    struct ext2_bgdesc      bg;         // 缓存唯一的块组描述符（从块2读取）
    uint                    dev;        // 设备号，传给 bread()/bwrite()
    uint                    inode_size; // 磁盘inode大小（128或256字节）
    struct sleeplock        lock;       // 串行化 balloc/ialloc 位图操作
    struct mount             *mnt;       // 反向指针，vnode → mp → mnt
};
```

各字段职责：

字段	来源	用途
sb	bread(dev, 1)	全局用：s_free_blocks_count、s_free_inodes_count、s_blocks_count
bg	bread(dev, 2)	定位：bg_inode_table、bg_block_bitmap、bg_inode_bitmap
inode_size	sb.s_inode_size（若为0则默认128）	ino_block/ino_offset 自适应计算
lock	初始化睡眠锁	保证bitmap_alloc/bitmap_free 的原子性
mnt	ext2 mount 填充	vnode通过priv->mp->mnt 回找挂载结构

3.4.2 ext2 vnode priv — vnode级私有数据

每个ext2 vnode附着一个 `ext2_vnode_priv`，挂载在 `vnode->priv` 上：

```

struct ext2_vnode_priv {
    struct ext2_inode_disk ino;           // 磁盘inode的内存缓存
    struct ext2_mount_priv *mp;          // 所属挂载的私有数据（用于bread/balloc等）
    uint inum;                           // 本inode的inode号
    int dirty;                           // 1 = ino已修改，需要在destroy时回写
    int orphaned;                        // 1 = unlink已将nlink降为0，destroy时真正释放
};

```

dirty标记的维护规则：

- `ext2_update_time` 更新 `atime/mtime/ctime` 时置1
- `ext2_bmap` 分配新块（间接块或数据块）时置1
- `ext2_i_truncate` 清空所有块指针和 `i_blocks / i_size` 后置1
- `ext2_create` 中 `links_count` 变化时置1
- `ext2_write` 里 `i_size` 扩展时置1
- `ext2_destroy` 中若非 `orphaned` 且 `dirty`，则 `write_inode` 回写磁盘

orphaned标记（见3.7.5节详解）：`unlink`将 `links_count` 降为0后置1，`ext2_destroy` 检测到此标记时执行块截断和inode释放。

3.5 块分配与回收（balloc/bfree/ialloc/ifree）

3.5.1 位图管理：bitmap_alloc / bitmap_free

块位图和inode位图均为一个1024字节块，通过以下两个通用函数管理：

```

// 在位图块 bb 中分配bit [1..nbits-1]（bit 0保留）
static int bitmap_alloc(mp, bb, nbits)
1. bread(mp->dev, bb)
2. acquiresleep(&mp->lock)           // 串行化并发分配
3. for i in 1 .. min(nbits, BSIZE*8):
    if data[i/8] 的第(i%8)位 == 0:
        data[i/8] |= (1 << (i%8))    // 置位
        bwrite(bp)
        releasesleep(&mp->lock)
    return i
4. releasesleep(&mp->lock); brelse(bp); return -1 // 位图满

// 在位图块 bb 中释放bit n
static void bitmap_free(mp, bb, n)
1. bread → acquiresleep → 清零目标位 → bwrite → releasesleep

```

位图使用睡眠锁：位图更新可能调用**bwrite**并等待磁盘I/O。**sleeplock**在保持分配操作互斥的同时允许I/O等待。

3.5.2 块分配流程（ext2_balloc）

```

ext2_balloc(mp):
1. b = bitmap_alloc(mp, bg.bg_block_bitmap, sb.s_blocks_count)
2. acquiresleep(&mp->lock)
3. mp->sb.s_free_blocks_count--           // 超块空闲块计数 -1
4. ext2_sb_write(mp)                     // 回写超块 (bread → memmove → bwrite)
5. releasesleep(&mp->lock)
6. return b

```

3.5.3 块释放流程 (ext2_bfree)

```

ext2_bfree(mp, blockno):
1. bitmap_free(mp, bg.bg_block_bitmap, blockno)
2. acquiresleep → sb.s_free_blocks_count++ → ext2_sb_write → releasesleep

```

3.5.4 inode分配 (ext2_ialloc)

```

ext2_ialloc(mp, mode):
1. inum = bitmap_alloc(mp, bg.bg_inode_bitmap, sb.s_inodes_count)
2. bread 该 inode 所在的块
3. memset(ino, 0, sizeof(ext2_inode_disk)) // 清零所有字段
4. ino->i_mode = mode                       // 0x4000(目录) / 0x8000(文件) /
0x2000(设备)
5. ino->i_atime = ino->i_mtime = ino->i_ctime = ticks
6. bwrite + brelse
7. acquiresleep → sb.s_free_inodes_count-- → ext2_sb_write → releasesleep
8. return inum

```

失败回滚：步骤2 bread失败 → 回滚 `bitmap_free(inode_bitmap, inum)`

3.5.5 inode释放 (ext2_ifree)

```

ext2_ifree(mp, inum):
1. bitmap_free(mp, bg.bg_inode_bitmap, inum)
2. acquiresleep → sb.s_free_inodes_count++ → ext2_sb_write → releasesleep

```

3.6 bmap：逻辑块→物理块映射

bmap是ext2所有I/O操作的必经之路，它将文件相对于块偏移量（`lbn`）转换为磁盘物理块号。

```

static uint ext2_bmap(struct ext2_vnode_priv *priv, uint lbn, int alloc)

```

参数	含义
priv	vnode私有数据，含inode缓存
lbn	文件内逻辑块号（0 = 第0块）
alloc	0 = 只读查找（read、stat、lookup用）；1 = 缺失时自动分配（write用）

寻址层次：

直接块区 (lbn 0..11):
ino->i_block[lbn] → 直接返回物理块号
若==0且alloc==1 → ext2_balloc → 写入i_block[lbn]
 i_blocks += 2 (一个1024字节块 = 两个512字节扇区)

单间接块区 (lbn 12..267):
1. 若 i_block[12]==0 且 alloc==1:
 a. ext2_balloc 分配间接块本身
 b. bread + memset(0, BSIZE) 清零整个间接块
 c. i_blocks += 2
2. bread(indirect_block) 读取间接块
3. indirect[lbn-12] → 目标数据块的物理块号
4. 若==0且alloc==1 → ext2_balloc → 写入indirect[lbn-12]
 i_blocks += 2
5. 返回物理块号

双重/三重间接(lbn 268+):
return 0 (不支持)

范围	块数	容量
直接块(0-11)	12	12KB
单间接块(12-267)	256	256KB
合计	268	268KB

bmap失败回滚：

- 分配间接块后 bread 失败 → ext2_bfree(间接块) → i_block[12]=0 → i_blocks -= 2
- 分配数据块后调用方 bread 失败 → bmap本身返回值非0，调用方检测

3.7 目录操作详解

3.7.1 目录项格式

ext2目录是一个由变长 ext2_dir_entry 组成的线性列表，散布在目录的数据块中：

```
struct ext2_dir_entry {  
    uint    inode;        // inode号（0 = 已删除的空位）  
    ushort  rec_len;      // 本条目总长度（含padding），必须被4整除  
    uchar   name_len;     // 真正名字长度（不含NUL）  
};
```

```

uchar  file_type;    // EXT2_FT_* 类型提示（避免额外读inode）
char   name[];       // 变长名字（长度 = name_len），对齐到rec_len
};

```

关键规则：

- `rec_len` 包含名字后的空余字节（padding），不允许条目跨块
- 第一个块的第一个条目以完整的 `rec_len = BSIZE` 开始
- 一个块末尾处若剩余空间不够最小条目（8字节），前一条目的 `rec_len` 会延伸到块尾
- `file_type` 为快速类型判断提供提示：`EXT2_FT_REG=1`、`EXT2_FT_DIR=2`、`EXT2_FT_CHRDEV=3`

3.7.2 ext2_dir_lookup 查找

```
static int ext2_dir_lookup(struct vnode *dir, char *name, struct vnode **result)
```

逐块扫描目录内容，对每个活跃条目（`inode != 0`）比较名字：

```

while(off < i_size):
    blk = ext2_bmap(dp, off/BSIZE, 0)    // 不分配
    bread(blk) → 逐条目:
        if name匹配:
            inum = de->inode
            *result = ext2_iget(mp, inum)    // 读inode → 创建vnode（持有ref）
            return 0
        off += de->rec_len
return -1                                // 未找到

```

性能特点： $O(N)$ 线性扫描——对于教学操作系统的目录规模（通常 <100 个条目）足够快。

3.7.3 ext2_dir_add 插入

```
static int ext2_dir_add(struct vnode *dir, char *name, uint inum, uchar ftype)
```

需求长度计算：`needlen = (sizeof(de) + strlen(name) + 3) & ~3`（4字节对齐）

三种插入策略（按优先级）：

情况1：复用已删除条目

de(v0)	(空闲空间, reclen >= needlen)
inode=0	

处理：直接填充 `inode`、`name_len`、`file_type`、`name` → `bwrite`

效果：原条目的 `rec_len` 不变

情况2: 拆分条目padding

de (活跃)	(padding, $\geq$ needlen)
real_len	reclen - real_len

处理:

1. 原条目 rec_len 缩小为 real_len
2. 在 real_len 偏移处创建新条目
3. 新条目 rec_len = 原reclen - real_len
4. 填充新条目 $\rightarrow$ bwrite

效果: 原活跃条目不变, 其padding变为新活跃条目

情况3: 扩展目录 (新建一个块)

在目录末尾 expand one data block:

ext2_bmap(dp, off/BSIZE, 1) $\rightarrow$ new block

memset(bp->data, 0, BSIZE)

de = first entry; de->rec_len = BSIZE (单个条目占满整块)

填充 inode / name $\rightarrow$ bwrite

ino->i_size = off + BSIZE (扩展目录大小)

情况2处理目录项内部空闲空间: ext2的rec_len可以大于实际占用空间。当目录项名称较短 (如a.c占16字节而rec_len=32) 时, 其余16字节为可回收padding, 新目录项可使用拆分后的空间。

3.7.4 ext2_create 创建文件

```
static int ext2_create(struct vnode *dir, char *name, short type, struct vnode
**new)
```

完整流程:

1. ext2_update_time(dp, 0, 1) // 目录mtime/ctime
2. ext2_dir_lookup(dir, name, &exist) // 不可重名
if 找到 $\rightarrow$ vput(exist); return -1
3. inum = ext2_ialloc(mp, mode) // 分配inode, mode = 0x8000(文件)或0x4000(目录)
if 失败 $\rightarrow$ return -1
4. read_inode $\rightarrow$ ino.i_links_count = 1 $\rightarrow$ write_inode
5. ext2_dir_add(dir, name, inum, ftype) // 在父目录插入条目
if 失败 $\rightarrow$ ext2_ifree $\rightarrow$ return -1
6. 若 type == V_DIR (目录):
 - a. newdir = ext2_iget(mp, inum) // 获取新目录vnode
 - b. ext2_dir_add(newdir, ".", inum, FT_DIR) // 自引用
 - c. ext2_dir_add(newdir, "..", dp->inum, FT_DIR) // 父目录引用
 - d. ndp->ino.i_links_count = 2 // "."和".."各计1个链接
 - e. ndp->dirty = 1; vput(newdir)


```
f. dp->ino.i_links_count++           // 父目录因".."获得一个额外链接

7. if new != NULL:
    *new = ext2_iget(mp, inum)        // 返回新vnode (ref持有)
```

失败回滚：步骤6中 "." 或 ".." 创建失败 → 手动回滚：清除父目录中刚插入的目录条目（将 `inode` 置0并 `bwrite`）+ `ext2_ifree` 释放 `inode`

3.7.5 ext2_unlink 删除与orphaned延迟释放

```
static int ext2_unlink(struct vnode *dir, char *name)
```

流程：

```
1. ext2_update_time(dp, 0, 1)           // 目录mtime/ctime

2. 扫描目录找到目标条目 → 获取 inum

3. read_inode → 判断 is_dir = (i_mode & 0xF000) == 0x4000

4. 若 is_dir:
    a. ext2_iget → 扫描目录内容，确认只有 "." 和 ".."
    b. 若非空 → return -1 (不能删除非空目录)
    c. dp->ino.i_links_count--           // 父目录失去".."的链接

5. de->inode = 0 → bwrite                // 标记目录条目为空

6. ino.i_links_count--
   若 links_count == 0:
       // orphaned延迟释放
       victim = ext2_iget(mp, inum)
       vp->orphaned = 1                  // 标记为孤立
       write_inode(mp, inum, &ino)       // 回写更新后的links_count
       vput(victim)                      // 此时 ref >= 1 (至少这里和可能的打开fd)
       // → 若ref降为0, destroy()中检测到 orphaned, 执行真正释放
   否则:
       ino.i_ctime = ticks; write_inode  // 仅更新时间戳
```

orphaned延迟释放的设计思想：

场景：文件被进程A打开，同时进程B unlink它

时间轴：

```
T1: open(fd, "/mnt/f") → vnode ref=2 (fd表 + namei临时)
T2: namei 退出         → vnode ref=1 (仅fd表)
T3: unlink("/mnt/f")   → links_count降为0, orphaned=1
                        → 此时不能立即释放块，因为fd表还持有vnode引用
T4: close(fd)          → vput → ref=0
```

```
→ destroy() 检测到 orphaned==1
→ ext2_i_truncate(priv) // 释放所有数据块
→ ext2_ifree(mp, inum) // 释放inode
```

实现结果：

- 安全的并发语义：open的文件在close前始终可读写，即使已被unlink
- 避免悬垂指针：links_count 降为0但vnode存在期间，inum 仍在inode位图中标记为已用
- 与Linux语义一致：符合POSIX "删除最后目录项时释放空间"的要求

3.8 时间戳管理

ext2维护三个POSIX时间戳（以 ticks 为单位；HAL统一为10Hz tick，即1 tick ≈ 100ms）：

```
static void ext2_update_time(struct ext2_vnode_priv *priv, int access, int mod)
```

参数	更新字段	调用场合
access=1, mod=0	i_atime	read、lookup、read_kernel
access=0, mod=1	i_mtime, i_ctime	write、create、unlink、truncate
access=1, mod=1	i_atime, i_mtime, i_ctime	（未使用）

调用方分布在10个vnode_ops函数中：

- ext2_read / ext2_read_kernel → ext2_update_time(priv, 1, 0)（访问时间）
- ext2_write → ext2_update_time(priv, 0, 1)（修改时间）
- ext2_lookup → ext2_update_time(dp, 1, 0)（目录atime）
- ext2_create / ext2_unlink / ext2_link / ext2_mknod → ext2_update_time(dp, 0, 1)（目录mtime）

3.9 设备节点支持

设备节点的 major/minor 号编码存储在 i_block[0] 中：

```
// 写入（ext2_mknod）：
ino->i_block[0] = (major << 8) | (minor & 0xFF);

// 读出（ext2_vnode_alloc）：
if(vp->type == V_DEV){
    vp->major = (ino->i_block[0] >> 8) & 0xFF;
    vp->minor = ino->i_block[0] & 0xFF;
}
```

设备节点类型为 EXT2_INODE_CHRDEV（字符设备，i_mode 高4位 = 0x2000），i_links_count = 1，无数据块。创建时通过 ext2_ialloc(mp, 0x2000) 分配inode。

3.10 限制与兼容性

限制类别	具体限制	原因
寻址	最大文件 268KB（直接+单间接）	未实现双重/三重间接块
文件系统大小	≤10MB（1个块组）	仅实现单块组，s_blocks_per_group 上限8192
块大小	仅1024字节	与xv6 BSIZE对齐，不支持2048/4096字节块
inode大小	128或256字节	动态检测s_inode_size，但仅操作前128字节
目录索引	线性扫描 O(N)	未实现HTREE索引目录
日志	无ext3/ext4日志	ext2本身无日志，崩溃恢复由上层负责
扩展属性	不支持	i_file_acl 忽略
ACL	不支持	i_dir_acl 忽略
符号链接	以普通文件处理	EXT2_INODE_SYMLNK 映射为 V_FILE
其他inode类型	FIFO、SOCK、BLKDEV 以普通文件处理	测试套件未覆盖

3.11 代码量统计

文件	行数	说明
kernel/ext2.h	128行	磁盘结构定义（superblock、bgdesc、inode_disk、dir_entry）+ 常量
kernel/ext2.c	1070行	全部逻辑实现（mount/umount + 13个vnode_ops + bmap + 分配器 + 目录操作 + 时间戳）
合计	1198行	

各模块代码量分解：

模块	函数	行数
mount/umount	ext2_mount, ext2_unmount, ext2_root	~80
vnode管理	ext2_vnode_alloc, ext2_destroy, ext2_iget	~80
inode I/O	ino_block, ino_offset, read_inode, write_inode	~40
位图分配器	bitmap_alloc, bitmap_free	~30
块/inode分配回收	ext2_balloc, ext2_bfree, ext2_ialloc, ext2_ifree	~70
bmap	ext2_bmap	~60
目录操作	ext2_dir_lookup, ext2_dir_add, ext2_create, ext2_unlink, ext2_mkdir, ext2_link, ext2_mknod	~280
读写	ext2_read, ext2_write, ext2_read_kernel	~90
truncate	ext2_i_truncate, ext2_truncate	~35
stat/readdir/时间戳	ext2_stat, ext2_readdir, ext2_update_time	~70

模块	函数	行数
vtable	ext2_vnops, ext2_vfsops	~30

第四章 xv6fs胶水层

4.1 设计原则：最小侵入

xv6fs胶水层的核心哲学是**零修改包装**——将xv6原生的 `struct inode` 及其操作函数（`fs.c` 中的 `iget/iput/readi/writei/dirlookup/dirlink/ialloc/itrunc/stati`）包装为VFS标准接口 `vnode_ops`，而不修改原生文件系统的任何一行代码。

原则	具体体现
fs.c 一行不改	原生文件系统代码（ <code>kernel/fs.c</code> 、 <code>kernel/log.c</code> 、 <code>kernel/fs.h</code> ）完全不动
薄适配层	<code>xv6fs.c</code> 约499行，每个ops函数平均约38行
日志语义保持	<code>begin_op/end_op</code> 包裹所有修改操作，与原始 <code>sysfile.c</code> 行为一致
引用计数透传	xv6的 <code>iget/iput</code> 引用计数语义不变， <code>vnode</code> ref通过 <code>xv6fs_vnode_priv</code> 间接持有 <code>inode</code>
不加新锁	继续使用xv6的 <code>ilock/iunlock</code> ， <code>vnode</code> 锁由VFS层在进入ops之前获取

4.2 inode→vnode 映射

4.2.1 私有数据结构

```
struct xv6fs_vnode_priv {
    struct inode *ip; // 指向xv6原生inode（已持有引用）
};
```

`xv6fs_vnode_priv`只保存原生`inode`指针。`type`、`major`、`minor`、`inum`和`size`等元数据继续由`struct inode`维护。

4.2.2 vnode工厂函数

```
static struct vnode* xv6fs_vnode_from_inode(struct inode *ip, uint dev, struct mount *mp)
```

字段映射表：

vnode字段	来源	说明
type	ip->type	T_DIR → V_DIR, T_DEVICE → V_DEV, 其他 → V_FILE
major	ip->major	设备主设备号
minor	ip->minor	设备次设备号

vnode字段	来源	说明
dev	参数dev	从上级vnode或挂载结构继承
mp	参数mp	所属挂载结构
inum	ip->inum	xv6 inode号（1 = ROOTINO）
size	ip->size	文件大小缓存
priv	新分配的xv6fs_vnode_priv	持有ip 的引用
ops	&xv6fs_vnops	指向xv6fs的vnode_ops虚表

调用前必须持有 **ilock(ip)**：因为 `ip->type`、`ip->major`、`ip->minor`、`ip->size` 的读取需要锁保护。工厂函数在读取完这些字段后调用方释放锁。

4.2.3 引用计数协议



4.3 各ops实现要点

ops	实现策略	行数	关键细节
lookup	<code>ilock(dp) → dirlookup(dp, name, 0) → iunlock(dp) → xv6fs_vnode_from_inode</code>	~12	dirlookup返回的inode已持有引用，工厂函数消费
read	<code>ilock(ip) → readi(ip, 1, buf, off, n) → iunlock(ip)</code>	~8	1 = 用户空间地址（readi内部调用copyout）
write	大写入拆分多个begin_op/ilock/writei/iunlock/end_op 迭代	~25	单次writei受(MAXOPBLOCKS-1-1-2)/2*BSIZE 日

ops	实现策略	行数	关键细节
			志配额限制，需循环
stat	<code>ilock → stati → iunlock → copyout</code>	~8	stati是xv6的串行化stat填充函数
readdir	<code>ilock → readi(ip, 0, &de, off, 14) → iunlock → 转换为vdirent → copyout</code>	~25	跳过 <code>inum==0</code> 的已删除条目；返回下一偏移
create	<code>begin_op/ilock/dirlookup检查重名/ialloc/dirlink/建"."/".."/iunlock/end_op</code>	~60	完整回滚逻辑：若"."或".."创建失败则清零父目录条目
unlink	<code>begin_op/ilock/dirlookup/isdirempty检查/ip->nlink--/iupdate/iunlockput/清零条目/iunlock/end_op</code>	~45	目录删除要求 <code>isdirempty</code> 通过； <code>iunlockput</code> 同时释放锁和引用
mkdir	委托 <code>xv6fs_create(dir, name, V_DIR, &new)</code>	~5	创建后立即 <code>vput(new)</code> （VFS层持有自己的ref）
link	<code>begin_op/ilock(old)/ip->nlink++/iupdate/iunlock/ilock(dp)/dirlink/iunlock/end_op</code>	~25	失败时回滚 <code>ip->nlink--</code>
mknod	<code>begin_op/ilock/dirlookup检查重名/ialloc(T_DEVICE)/设major+minor/iupdate/dirlink/iunlock/end_op</code>	~30	-
read_kernel	<code>ilock → readi(ip, 0, buf, off, n) → iunlock</code>	~8	<code>0</code> = 内核空间（直接memmove），exec加载ELF用
truncate	<code>begin_op/ilock/itrunc/iunlock/end_op</code>	~8	itrunc释放所有数据块， <code>vp->size = 0</code>
destroy	<code>begin_op/iput(ip)/end_op/kfree(priv)</code>	~10	iput含日志包裹，可能触发的itrunc/ifree也在日志内

4.4 代码量

文件	行数	说明
kernel/xv6fs.c	499行	全部胶水逻辑实现
模块	行数	
数据结构与vnode工厂	~40	
mount/root	~35	
destroy（释放回调）	~12	
vnode_ops（13个函数）	~210	
vtable（虚表填充）	~20	

注：xv6fs.c通过VFS标准接口vnode_ops + vfs_ops导出功能，相关类型定义位于vfs.h，当前未设置独立头文件。

第五章 HAL设计概述

5.1 设计目标与抽象边界

HAL（Hardware Abstraction Layer）的目标是让通用内核只依赖**稳定的操作语义**：开关中断、建立地址空间、进入/离开用户态、收发字符、收取外设中断、启动定时器和提交块请求。寄存器编码、页表格式、中断控制器协议和设备模型全部留在平台目录内部。

最终边界为：

kernel/：进程、调度、系统调用、VFS、buffer cache、通用trap策略

| 只调用 hal_* 接口或使用平台提供的基础类型/常量

▼

hal/hal.h + 7 个 hal_*.h：平台无关的语义契约

| Makefile通过 -Ihal/\${ARCH} 和 ARCH_OBJS 在编译期选择一套实现

| — hal/riscv/：RV64、Sv39、PLIC、SSTC、virtio-mmio

| — hal/loongarch/：LA64、四级页表、软件TLB重填、PCH-PIC/EIOINTC、CSR定时器、RAM disk

当前kernel/中没有ARCH_riscv或ARCH_loongarch条件块，原先直接使用的intr_*、r_sstatus、w_stvec等旧名字也已收敛为hal_*调用。平台差异位于HAL内部和Makefile的构建选择中。

5.1.1 编译期平台选择与设备数量

HAL采用编译期静态绑定。一份内核镜像对应一个CPU架构，Makefile根据ARCH选择工具链、平台对象、链接脚本、头文件搜索路径和QEMU参数；设备数量统一由HAL_NDISK控制。VFS按运行期挂载需求使用注册表和函数表。两类选择机制分别对应“单平台内核镜像”和“多文件系统并存”两种模型。

```
ARCH ?= riscv
HAL_NDISK ?= 3
CFLAGS += -DARCH_${ARCH}
CFLAGS += -DHAL_NDISK=${HAL_NDISK}
```

```
CFLAGS += -Ihal/${ARCH}

HAL_COMMON_OBJS = hal_entry.o hal_start.o hal_swch.o \
                  hal_tramp.o hal_kvec.o hal_uart.o hal_vm.o

# RISC-V: + hal_plic.o hal_virtio.o
# LoongArch: + hal_intr.o hal_ramdisk.o hal_tlbrefill.o
```

hal/hal.h包含平台arch.h/memlayout.h和全部子系统接口。-Ihal/\${ARCH}使同名平台头文件在编译期被选中；K_OBJS双架构完全一致，平台差异只体现在ARCH_OBJS和链接脚本。

维度	HAL	VFS
抽象对象	CPU、MMU、中断控制器和平台设备	同时存在的多个文件系统
选择时机	编译期，ARCH=riscv/loongarch	运行期，mount时选择后端
调用代价	普通函数或内联函数，无间接调用	vnode_ops函数指针
数量配置	HAL_NDISK=1..3编译期常量	运行期注册任意多个FS
新增实现	增加平台目录并实现全部契约	注册新的vfs_ops/vnode_ops
通用层条件块	kernel/中为0	上层不判断具体文件系统

5.1.2 接口按职责划分

当前接口为hal.h + 7 个 hal_*.h，共234行。hal_memlayout.h空壳已删除，块设备接口独立为hal_disk.h。

接口	主要契约	通用层调用者
hal_arch.h	hart ID、中断开关、异常状态、异常向量、页表根、CPU idle、绑定用户trapframe	proc.c、trap.c
hal_vm.h	TLB刷新、叶PTE判断、内核映射、MMU使能、trampoline/trapframe特殊映射	vm.c、proc.c
hal_intr.h	中断控制器全局/每核初始化、claim（-1表示无IRQ）、complete	main.c、trap.c
hal_timer.h	每核初始化hal_timer_init()、当前tick确认并安排下一tick hal_timer_ack()	main.c、trap.c
hal_console.h	初始化、批量输出、同步单字符输出、中断处理（TX empty + RX FIFO排空）	console.c、trap.c、printf.c
hal_disk.h	HAL_NDISK、hal_disk_init/rw/intr	bio.c、main.c、trap.c
hal_ctx.h	ABI相关的struct hal_context、字段偏移断言和hal_switch()	proc.c
hal.h	聚合平台头文件和全部接口	所有内核模块

5.1.3 边界设计原则

1. **接口表达操作语义。** 通用层调用`hal_vm_enable()`；RISC-V写`satp`，LoongArch配置PGDL/PGDH/DMW/PWCL/PWCH/TLBRENTY。寄存器名字不出现在`kernel/`。
2. **平台空操作保持公共生命周期。** `hal_vm_map_trampoline()`在RISC-V建立TRAMPOLINE/TRAPFRAME映射，在LoongArch返回成功；调用方总是执行“创建/检查/释放”完整流程。
3. **每项硬件职责保留一条执行路径。** 删除旧PLIC/UART/swtch包装、无人调用的`hal_console_poll()`和C语言手工TLB填充；平台文件直接实现公共接口。
4. **返回值和数量统一。** `hal_irq_claim()`两平台都以-1表示无中断，因此LoongArch的IRQ 0是合法设备号；磁盘数量统一为`HAL_NDISK`，`kernel/bio.c`不再出现`VIRTIO_NDISK`。
5. **ABI和布局用编译期断言保护。** `struct hal_context`、`struct trapframe`的字段偏移和总大小均有`_Static_assert`；LoongArch内核栈窗口和`.tlbrefill`大小也有C或链接期断言。改动C结构而忘记改汇编会直接编译失败。

5.2 公共内核调用关系

5.2.1 启动后的统一初始化顺序

两个平台的入口和特权级准备不同，但进入`kernel/main.c`后共用同一条初始化链：

CPU0:

```
consoleinit()/printfinit()
→ kinit()
→ kvminithart()           // hal_vm_map_kernel()
→ kvminithart()           // hal_vm_enable()
→ procinit()              // 映射每个进程的两页内核栈
→ trapinit()/trapinithart()
→ hal_irq_init()/hal_irq_hart_init()
→ binit()/iinit()/vfs_init()/fileinit()
→ hal_disk_init()
→ hal_timer_init()
→ userinit()
→ scheduler()
```

AP:

```
等待 started
→ kvminithart()
→ trapinithart()
→ hal_irq_hart_init()
→ hal_timer_init()       // 7c349d3: AP 也建立本地定时器
→ scheduler()
```

`consoleinit()`内部调用`hal_console_init()`。CPU0执行全局设备初始化；每个AP在进入调度器前都使能本核页表、trap向量、外部中断和本地定时器。这保证AP在`wfi`空闲后仍能被定时器唤醒，并且运行在AP上的用户进程具有时间片。

5.2.2 页表和用户地址空间生命周期中的平台职责

HAL把最初的10个架构条件块收敛为按资源生命周期组织的接口：

原条件差异	最终接口	RISC-V	LoongArch
返回用户态前保存trapframe位置	<code>hal_trap_bind_user_frame</code>	无额外动作	将trapframe内核VA写入KSave1
创建用户页表特殊映射	<code>hal_vm_map_trampoline</code>	映射TRAMPOLINE和TRAPFRAME	空实现
释放用户页表特殊映射	<code>hal_vm_unmap_trampoline</code>	解除映射	空实现
建立/启用内核地址空间	<code>hal_vm_map_kernel/enable</code>	建Sv39映射并写satp	DMW低地址直映，配置PGDL/PGDH与软件重填
<code>freewalk</code> 识别叶项	<code>hal_pte_is_leaf</code>	检查R/W/X	检查LoongArch P位
<code>mappages</code> 权限翻译	<code>hal_pte_encode_perm</code>	原样返回	R/X软件位翻译为NR/NX

`proc_pagetable()`现在检查`hal_vm_map_trampoline()`的返回值，失败时释放页表并返回0，避免产生缺少特殊映射的残缺进程。

5.2.3 上下文切换与页表根

`hal_switch(old, new)`保存`ra`、`sp`和ABI规定的callee-saved寄存器：RISC-V为14个（`ra, sp, s0..s11`），LoongArch为12个（`ra, sp, fp, s0..s8`）。`hal_ctx.h`对两个结构分别断言字段偏移和总大小，保证C布局与`hal_switch.S`逐条`st/ld`完全一致。

从进程切到调度器时，`p->context.sp`指向该进程的内核栈，`cpu->context.sp`指向每CPU调度器栈。所有进程内核栈均映射在共享的`kernel_pagetable`中，因此切换时保持`satp/PGDH`不变。

5.3 公共接口的执行语义

5.3.1 CPU和trap状态

`hal_get_hartid()`返回当前核编号；`hal_intr_on/off/get()`管理当前CPU的全局中断使能；`hal_read/write_sstatus`、`hal_read/write_sepc`、`hal_read/scause/stval`向公共trap代码提供异常状态；`hal_write_stvec()`安装异常入口；`hal_cpu_idle()`在调度器空闲路径执行平台等待指令。

RISC-V实现直接访问S态CSR。LoongArch实现把PRMD、ERA、ESTAT和BADV转换为公共`trap.c`使用的状态格式：`SSTATUS_SPP/SPIE`由PRMD的PPLV/PIE转换，`scause`由`estat_to_scause()`统一映射，`sepc/stval`对应ERA/BADV。

5.3.2 虚拟内存

`hal_vm_map_kernel()`建立平台固定映射，`hal_vm_enable()`激活当前CPU的内核地址空间。`hal_vm_map_trampoline()`和`hal_vm_unmap_trampoline()`处理每个用户页表的特殊映射。`hal_pte_is_leaf()`为`freewalk()`提供叶PTE判定。用户地址下界不再引入平台专用HAL接口，两架构均从VA 0装载用户ELF。

通用`walk()`通过`PT_LEVELS`、`PX()`、`PA2PTE()`和`PTE2PA()`处理平台页表。`mappages()`在写入叶PTE时调用`hal_pte_encode_perm()`，把公共R/W/X/U请求转换为平台原生权限；`uvmcopy()`通过`PTE_FLAGS()`/`hal_pte_decode_perm()`恢复公共权限。

5.3.3 中断、定时器和控制台

`hal_irq_init()`初始化共享中断控制器，CPU0调用一次；`hal_irq_hart_init()`设置每核中断接收状态。`hal_irq_claim()`返回待处理IRQ，**两平台统一以-1表示无中断**，因此LoongArch的UART IRQ 0是合法向量；`hal_irq_complete()`执行控制器完成协议。RISC-V PLIC在HAL内把硬件返回值0转换成-1。

定时器接口只保留两个语义：`hal_timer_init()`初始化本核定时器，`hal_timer_ack()`确认当前tick并安排下一tick。`clockintr()`不再假设平台支持绝对deadline，也不再计算`time+1000000`。RISC-V和LoongArch都配置为10Hz tick；所有CPU各自确认定时器，仅CPU0累加全局ticks。

控制台接口分工如下：

接口	功能	公共调用路径
<code>hal_console_init()</code>	配置UART寄存器并初始化tx_lock	<code>consoleinit()</code>
<code>hal_console_write(buf,n)</code>	批量发送，TX busy时睡眠等待	<code>consolewrite()</code>
<code>hal_putchar(c)</code>	同步单字符输出；普通路径持tx_lock，panic路径绕锁	<code>printf</code> 、回显、panic
<code>hal_console_intr(handler)</code>	处理TX empty唤醒写者，排空RX FIFO并回调	<code>devintr()</code>

两个平台的UART现在使用同一并发协议：`tx_lock + tx_busy + sleep/wakeup + TX-empty IRQ`。`hal_putchar()`同步路径也持有tx_lock并设置tx_busy，避免多CPU同时写THR；仅在panicking路径故意绕锁，防止其他CPU持锁时panic输出永久阻塞。退格、Ctrl-P、换行提交、输入环形缓冲和进程唤醒仍由`kernel/console.c:consoleintr()`处理。

5.3.4 块设备

块设备接口集中在`hal/hal_disk.h`：

```
#define HAL_NDISK 3    // 可配置为 1..3
void hal_disk_init(void);
void hal_disk_rw(struct buf *b, int write);
void hal_disk_intr(void);
```

`kernel/bio.c`使用HAL_NDISK校验设备号，通过`hal_disk_rw()`填充或回写`struct buf`。RISC-V实现使用异步virtio队列，LoongArch实现使用同步RAM窗口复制；buffer cache观察统一的`b->disk`完成状态。

5.4 代码规模和演进记录

类别	文件/规模	内容
公共接口	<code>hal.h</code> + 7 个 <code>hal_*.h</code> ，234行	接口声明、上下文结构和偏移断言
RISC-V	13个文件，1608行	启动、VM、trap、switch、PLIC、UART、virtio、链接脚本和virtio头
LoongArch	14个文件，1643行	启动、VM、TLB refill、trap、switch、EIOINTC、UART、RAM disk和链接脚本

类别	文件/规模	内容
总计	35个文件，3485行	包含注释、头文件和链接脚本

RISC-V目录包含从原xv6迁移的平台代码，不能把3485行全部视为净新增；真正新增集中在LoongArch平台、接口层和少量RISC-V扩展。主要提交记录如下：

提交	修改内容	结果
c7c4a1c	QEMU loader装载三镜像	-bios仅装载内核，三幅镜像进入保留RAM窗口
abdbbb9	文件系统设备布局调整	dev=1、2、3分别对应xv6fs、ext2和FAT32
63bed2d	LoongArch内核栈保护	高地址栈、固定PGDH、未映射guard和紧急异常栈
0b463d3	删除C语言手工TLB填充	vmfault()建立PTE，汇编refill装入TLB
7bb6704	清理UART、PLIC和swtch包装	平台文件直接实现hal_*接口
c5cfd4b	LoongArch PTE权限转换	公共R/X请求转换为NR/NX原生权限
66acbd8	LoongArch UART初始化	LSR状态轮询和完整16550初始化；后续由中断驱动取代轮询
f08c90f、f731aed	HAL边界收敛	kernel/中的架构条件块清零
91cfd94	死代码和链接布局清理	RAM disk命名修正，增加链接和栈窗口断言
f94c185	UART/EIOINTC根因修复	EIOINTC改经IOCSR访问，HWI改IS[2]，恢复RX/TX中断
7c349d3	AP定时器与失败路径	AP初始化本地定时器；trampoline返回值检查；UART跨CPU串行化
a7f679c	接口与布局收口	hal_timer_init/ack、双平台10Hz tick、trapframe/context断言、hal_disk.h/HAL_NDISK、Makefile清理

第六章 RISC-V平台适配

6.1 平台文件与实现范围

RISC-V是原始xv6的基准平台。平台代码迁移到hal/riscv/后，旧转发函数被删除，hal_*函数直接承担原驱动职责。页表、trap、PLIC、UART和virtio的核心算法沿用MIT xv6实现，磁盘数量扩展为HAL_NDISK=1..3。

文件	实现内容
arch.h	CSR访问、Sv39 PTE宏、中断控制和HAL内联函数

文件	实现内容
<code>hal_entry.S</code> 、 <code>hal_start.c</code>	每核启动栈、M态初始化、SSTC授权、进入S态
<code>hal_vm.c</code> 、 <code>memlayout.h</code>	内核固定映射、satp激活、QEMU物理地址
<code>hal_tramp.S</code> 、 <code>hal_kvec.S</code>	用户和内核trap入口
<code>hal_swth.S</code>	进程与调度器上下文切换
<code>hal_plic.c</code>	PLIC优先级、使能、claim和complete
<code>hal_uart.c</code>	16550初始化、RX/TX中断和带锁同步输出
<code>hal_virtio.c</code> 、 <code>virtio.h</code>	1~3个virtio-mmio设备及请求队列
<code>kernel.ld</code>	内核段布局和trampoline页约束

6.2 CPU启动与特权级切换

QEMU通过`-kernel`把ELF内核加载到`KERNBASE=0x80000000`。`hal_entry.S`读取`mhartid`，计算`stack0 + (hartid+1)*4096`作为当前核栈顶，随后跳转到M态`start()`。`hal_start.c`依次完成：

1. 将`mstatus.MPP`设为S态，把`mepc`设为`main`；
2. 暂时关闭分页，委托异常和中断到S态；
3. 通过PMP允许S态访问物理内存；
4. 打开SSTC扩展及`time/stimecmp`访问权限，设置首次定时中断；
5. 把hart ID写入`tp`，执行`mret`进入公共`main()`。

QEMU virt的timebase为10MHz。`TIMER_TICK_CYCLES`固定为`1000000`，即一个xv6 tick为100ms（10Hz）。M态`timerinit()`完成`menvcfg/mcounteren`授权并写首次`stimecmp`；S态`hal_timer_init()`和`hal_timer_ack()`继续以相同周期写`stimecmp`。

6.3 Sv39内核与用户地址空间

RISC-V使用三级Sv39页表，虚拟页号按9-9-9分级，页内偏移12位。`PA2PTE`把物理页号放到PTE `[53:10]`，低位保存V/R/W/X/U/G/A/D标志。通用`mappages()`提供正向的PTE_R/PTE_W/PTE_X/PTE_U权限，RISC-V硬件直接解释这些位。

```
VA[38:30] L2 | VA[29:21] L1 | VA[20:12] L0 | VA[11:0] offset
PTE[53:10] PPN | PTE[9:0] flags
```

`PT_LEVELS=3`使公共`walk()`从level 2遍历到level 0。目录PTE包含V和下一级页表物理地址；`hal_pte_is_leaf()`检查R/W/X任一位。

`hal_vm_map_kernel()`建立UART、PLIC、`HAL_NDISK`个virtio-mmio窗口、只读可执行内核代码和可读写内核数据/RAM映射；`hal_vm_enable()`执行“刷新TLB→写satp→再次刷新TLB”。每个用户页表还通过`hal_vm_map_trampoline()`映射：

- **TRAMPOLINE**：用户页表和内核页表中虚拟地址相同，切换页表的汇编代码可连续执行；
- **TRAPFRAME**：保存用户寄存器以及`kernel_satp/kernel_sp/kernel_trap/kernel_hartid`；映射未设置PTE_U。

`exec()`从VA 0装载ELF段。释放进程页表时，`hal_vm_unmap_trampoline()`先解除TRAMPOLINE和TRAPFRAME映射，随后公共代码释放用户页和页表页。

6.4 用户态与内核态trap

用户程序

- `trap`: 硬件写`sepc/scause`，跳到`stvec=uservec`
- `uservec`: 用`sscratch`暂存`a0`，保存全部用户寄存器
- 从`trapframe`取`kernel_sp`、`kernel_trap`和`kernel_satp`
- 切内核栈和内核页表，调用`usertrap()`
- `prepare_return()`准备`sstatus/sepc`和`trapframe`
- `userret`: 切回用户`satp`，恢复寄存器，`sret`

`prepare_return()`把`stvec`设置为`TRAMPOLINE+(uservec-trampoline)`，并把内核页表根、内核栈顶、`usertrap`地址和hart ID写入`trapframe`。`uservec`使用`csrrw a0, sscratch, a0`暂存用户`a0`，保存全部用户寄存器后切换`sp`、`tp`和`satp`。

`prepare_return()`关闭中断，设置`sstatus.SPP=0`和SPIE，并写入`sepc`。`trampoline`中的`userret`写用户`satp`、执行`s fence.vma`、恢复用户寄存器并执行`sret`。

S态中断和异常进入`kernelvec`。汇编在当前内核栈保存caller-saved寄存器，调用`kerneltrap()`，恢复现场后执行`sret`。`kernel/proc.h`对`trapframe`全部字段偏移和总大小做了`_Static_assert`，防止汇编与C布局漂移。

6.5 PLIC与定时器

PLIC采用“priority、per-hart enable、threshold、claim、complete”协议。`hal_irq_init()`把UART IRQ 10和已配置virtio磁盘的IRQ优先级设为1。`hal_irq_hart_init()`在当前hart的S态enable寄存器中设置对应位，并把threshold设置为0。

设备置PLIC pending

- CPU收到S态外部中断
- `devintr()`
- `hal_irq_claim()`读取`PLIC_SCLAIM(hart)`
- `hal_console_intr()`或`hal_disk_intr()`
- `hal_irq_complete(irq)`写回`PLIC_SCLAIM(hart)`

`hal_irq_claim()`把PLIC返回的0（无中断）转换为-1，与LoongArch保持一致。定时器使用SSTC：

`hal_timer_init()`和`hal_timer_ack()`都写入`stimecmp = time + TIMER_TICK_CYCLES`。`clockintr()`只调用`hal_timer_ack()`，仅CPU0累加ticks。

6.6 16550 UART

`hal_console_init()`先初始化`tx_lock`，再关闭IER、设置DLAB和波特率除数3、配置8N1、复位并启用FIFO，最后打开RX和TX中断。

发送接口	同步机制	调用环境
<code>hal_console_write()</code>	<code>tx_lock</code> 保护； <code>tx_busy</code> 时进程在 <code>tx_chan</code> 睡眠；TX empty中断唤醒	控制台设备 <code>write()</code>

发送接口	同步机制	调用环境
<code>hal_putchar()</code>	非panic路径持有 <code>tx_lock</code> ，轮询LSR bit 5后写THR，置 <code>tx_busy=1</code>	<code>printf</code> 、输入回显
panic输出	<code>panicking!=0</code> 时绕锁直接轮询THR	避免其他CPU持锁导致panic死锁

`hal_console_intr()`读取IIR确认中断状态。LSR bit 5置位时，在`tx_lock`内清除`tx_busy`并唤醒发送进程；随后循环检查LSR bit 0，从RHR取字符并回调`consoleintr(c)`。接收路径由PLIC中断触发，不再存在定时器轮询路径。

6.7 virtio-mmio块设备

`HAL_NDISK`个virtio-blk设备依次位于`0x10001000`、`0x10002000`和`0x10003000`。`hal_virtio.c`为每个设备保存独立MMIO基址、virtqueue、描述符空闲状态、buffer等待信息和自旋锁。

`hal_disk_init()`逐盘校验magic、version和device ID，设置ACKNOWLEDGE与DRIVER状态，完成特性协商，登记descriptor、available ring和used ring地址，最后设置DRIVER_OK。

一次`hal_disk_rw()`使用三个描述符：

```
descriptor 0: virtio_blk_req, 包含读写类型和sector
descriptor 1: b->data, 长度BSIZE, 方向由请求类型决定
descriptor 2: 设备status字节
```

描述符链头加入available ring后，驱动写QueueNotify，设置`b->disk=1`，并在设备锁保护下执行`sleep(b, &lock)`。`hal_disk_intr()`读取used ring，对完成项清除`b->disk`、释放描述符链并`wakeup(b)`。`dev=1、2、3`分别承载xv6fs、ext2和FAT32。`HAL_NDISK=1/2/3`均可编译运行。

6.8 上下文切换和数据通路

`hal_swch.S`按照RISC-V ABI保存`ra`、`sp`和`s0`至`s11`。`sched()`调用`hal_switch(&p->context, &cpu->context)`进入调度器；`scheduler()`调用反向切换恢复进程。新进程的`context.ra`为forkret，`context.sp`为两页内核栈顶。`hal_ctx.h`断言该结构大小为112字节。

`dev=2`的ext2读取路径为：

```
ext2_read() → bread(2, blockno) → hal_disk_rw(b, 0)
    → virtio描述符链 → QueueNotify → sleep
    → PLIC磁盘中断 → hal_disk_intr() → wakeup
    → ext2解析b->data
```

第七章 LoongArch平台适配

7.1 平台文件与实现范围

LoongArch实现面向QEMU 8.2.2 virt机器和la464 CPU，使用LA64指令集、PLV0/PLV3特权级、四级页表、软件TLB重填、PCH-PIC/EIOINTC和16550 UART。UART收发已恢复为与RISC-V一致的中断驱动模型。

文件	实现内容
arch.h	CSR编号、异常码、PTE格式、权限转换、IOCSR访问和trap状态转换
hal_entry.S、hal_start.c	每核启动栈、flash到RAM搬运、多核同步和10Hz定时器
hal_vm.c、memlayout.h	DMW0、PGDL/PGDH、高地址内核栈窗口和RAM disk地址
hal_tlbrefill.S	四级软件页表遍历和TLB填入
hal_tramp.S、hal_kvec.S	PLV3/PLV0异常入口、返回和栈溢出诊断
hal_swtrch.S	LoongArch ABI上下文切换
hal_intr.c	PCH-PIC mask解除和EIOINTC IOCSR配置
hal_uart.c	16550初始化、RX/TX中断和带锁同步输出
hal_ramdisk.c	三个loader-backed同步块设备
kernel.ld、user.ld	固件/RAM分离、TLB refill页断言和用户程序VA 0入口

7.2 启动、链接和内存搬运

LoongArch内核由QEMU -bios加载到0x1c000000的固件窗口，CPU从PLV0启动。链接脚本把不同阶段的代码固定在可验证的位置：

区域	地址	作用
.text.boot	0x1c000000起	最早期入口和start()；链接断言不得越过0x1c008000
.tlbrefill	0x1c008000	4KiB对齐的软件TLB重填入口；链接断言不超过4KiB
.trampoline	0x1c009000	固定一页的用户陷入/返回代码
普通.text/.rodata	0x1c00a000起	在flash中直接执行
.data/.bss/.bootstack	VMA 0x00400000起	运行时位于可写RAM；.data初值的LMA仍在flash

CPU 0把.data从LMA复制到RAM并清零.bss，通过内存屏障写入BOOT_DONE_MAGIC；其他CPU等待该标志后访问全局变量。启动阶段保持定时器关闭；公共trap向量安装后，CPU0和每个AP都在main()中调用hal_timer_init()，因此所有CPU都有本地10Hz定时器，AP空闲时可被唤醒，运行在AP上的进程也拥有时间片。

7.3 DMW0、PGDL、PGDH与四级页表

LoongArch采用四级9-9-9-9-12页表，PWCL/PWCH指定索引为VA [20:12]、[29:21]、[38:30]、[47:39]。硬件有效地址能力大于本项目用户空间，但MAXVA仍保持1<<38，从而保留xv6边界测试、TRAPFRAME和用户增长上限的原有契约。HAL_VA_BITS=38同时被C和汇编引用。

当前地址空间使用三条转换路径：

低地址、PLV0: DMW0恒等映射

内核代码、数据、UART、trampoline、loader RAM disk

低地址、PLV3: PGDL

当前进程用户页表; 返回用户态时切换

高地址、PLV0: 固定PGDH

全部进程的高地址内核栈及未映射guard页

LoongArch的`hal_vm_map_kernel()`为空函数: 内核代码、数据、UART和RAM disk等低地址对象全部由DMW0访问, 不建立对应的低地址页表项。公共`kvmmake()`仍按统一生命周期建立TRAMPOLINE页和高地址内核栈映射; 其中TRAMPOLINE页表项在DMW0下不会被查询, 保留它是为了让通用VM代码双架构零条件分支。

`hal_vm_enable()`设置PGDL和固定PGDH, 配置DMW0、PWCL/PWCH、RVACFG与TLBRENTY, 最后清除CRMD.DA并设置CRMD.PG。PGDL在用户返回和用户陷入时切换; PGDH在每个CPU启动分页时设置一次。

早期实现曾为MAXVApplus测试保留VA 0和0x1000两页, 并将用户程序改从0x2000链接。复核LoongArch地址合规检查后确认, 该保留区不是ISA或ABI要求, 且当前RVACFG=8、MAXVA=1<<38、TLB refill上界检查和C路径检查已完整拦截越界地址。因此最终删除两页PLV0保留映射、相关HAL接口和生命周期分支, 双架构用户ELF统一从VA 0装载。该清理使实现代码净减78行, 其中HAL目录减少66行。QEMU 8.2.2 LA464下的MAXVApplus与完整usertests-q均通过。

7.4 软件TLB重填

LA464在TLB miss后跳到`hal_tlbrefill.S`。重填入口运行在DA=1/PG=0环境, 使用纯汇编完成地址检查和页表遍历。入口把t0保存到TLBSAVE, 高地址检查使用t1时把原值保存到KSave2。

执行顺序为:

1. 把TLBREHI页大小明确设为12, 即4KiB;
2. 验证地址只属于用户[0, MAXVA)或高地址内核栈窗口[KSTACK_REGION_BOTTOM, KSTACK_TOP); 边界直接引用HAL_VA_BITS/KSTACK_*宏, 不再在汇编里复制38/-193/-1等魔法数;
3. 读取CSR.PGD, 让硬件根据TLBRBADV自动选择PGDL或PGDH;
4. 依次用lddir 3/2/1走三级目录, 并检查每一级V位; 确认有效后addi.d -1得到下一级页表基址;
5. ldpte 0/1同时装入偶页和奇页, 执行tlbfill;
6. 地址或PTE无效时填入匹配且V=0的条目。CPU重试后产生普通PIL/PIS/PIF异常, 进入标准trap路径。

早期版本在`vmfault()`中通过C函数`tlb_fill_from_pte()`手工填TLB。该函数把同一PTE写入偶、奇两个半项, 存在页别名风险, 且写入结果会在用户返回前被全量`invtlb`清除。提交0b463d3删除该路径。当前`vmfault()`只检查访问并建立PTE, `hal_tlbrefill.S`完成PTE到TLB的装入, 保证TLB填充只有一条执行路径。

7.5 PTE编码与权限转换

RISC-V和LoongArch分别定义PA2PTE/PTE2PA/PX。当前LoongArch配置把对齐物理地址保存在PTE的[PALEN-1:12]字段, 低位及高控制位保存属性。通用`walk()`通过平台宏访问页表, 不依赖具体PTE位布局。

LoongArch通过NR (Not Readable) 和NX (Not Executable) 表达读、执行限制。公共VM代码保留PTE_R/PTE_X作为软件请求位, 并在写叶PTE的位置调用`hal_pte_encode_perm()`:

位	名称	当前实现中的用途
0	V	页表项有效
1	D	页面可写状态的一部分
3:2	PLV	PLV0内核页或PLV3用户页
5:4	MAT	当前使用可缓存一致属性
6	G	全局映射
7	P	物理页存在，也是叶PTE判定依据
8	W	硬件页表可写位
61	NR	置位表示不可读
62	NX	置位表示不可执行
63	RPLV	受限特权检查

```
*pte = PA2PTE(pa) | hal_pte_encode_perm(perm) | PTE_V_CACHE;
```

`hal_pte_encode_perm()`保留公共请求中的PLV、G和写权限字段，并根据R/X请求设置NR/NX；外层`PTE_V_CACHE`提供V、MAT和P。由此生成的代码页具有读和执行权限，数据页及用户栈具有读写和NX权限，只执行页面通过NR=1、NX=0表示。`uvmcopy()`通过`hal_pte_decode_perm()`恢复公共权限并复制到子进程。

7.6 用户trap与异常语义转换

LoongArch低地址trampoline由PLV0的DMW0执行，用户页表未建立高地址TRAMPOLINE和TRAPFRAME映射。返回用户态前，`hal_trap_bind_user_frame()`把当前`p->trapframe`的内核地址写入KSave1：

PLV3异常

- EENTRY=uservec
- KSave0暂存用户a0，KSave1取得trapframe内核地址
- 保存用户寄存器，切到内核PGDL和kernel_sp
- usertrap()
- prepare_return()重新绑定KSave1
- userret切回用户PGDL、全量invtlb、恢复寄存器、ertn

`estat_to_scause()`把LA异常转换为通用`trap.c`使用的scause值：SYS→8，PIF→12，PIL→13，PIS→15，INE→2；ADE依据子码区分取指和访存地址错误；外部中断由`ESTAT_IS_HWI`（当前为IS[2]）映射为supervisor external interrupt。PRMD中的前一特权级和中断使能位转换为`sstatus`语义，ERA和BADV分别对应sepc和stval。

7.7 高地址内核栈与guard页

初版LoongArch内核栈位于低地址，虽然`proc_mapstacks()`为每个栈留出未映射页，但DMW0会绕过页表，栈下溢仍能写入物理内存。这种“物理间隔”只能降低相邻对象被立即覆盖的概率，不能检测越界。

当前实现采用页表缺页保护：

```

KSTACK_TOP
  进程0: 2页映射栈, 初始sp = KSTACK(0) + 2*PGSIZE
          1页无PTE guard
  进程1: 2页映射栈
          1页无PTE guard
  ...
KSTACK_REGION_BOTTOM = KSTACK_TOP - NPROC*3*PGSIZE

```

所有栈移到VA[63]=1的高地址窗口, 由固定PGDH映射; DMW0只覆盖低地址。KSTACK_REGION_BOTTOM按NPROC*3*PGSIZE自动计算, hal_vm.c中的_Static_assert把窗口大小与NPROC绑定。TLB refill允许该窗口并拒绝其他高地址。若栈进入guard页, 先产生无效TLB项, 再形成PIL/PIS。普通异常入口此时不能继续使用已经越界的sp, 因此hal_kvec.S先检查异常码和BADV, 在保存现场前切到每CPU启动/紧急栈, 再调用hal_stack_overflow_panic()稳定输出原SP和故障地址。

proc_mapstacks()为每个进程分配两个物理页, 从KSTACK(p)开始连续映射。allocproc()中的context.sp和prepare_return()写入trapframe的kernel_sp均为p->kstack+2*PGSIZE。guard页位于KSTACK(p)-PGSIZE, 页表中没有对应PTE。

该实现增加地址检查和紧急异常路径, 把栈越界转换为包含原SP和故障地址的确定性panic, 而不是静默破坏相邻进程栈。

7.8 EIOINTC、定时器和UART

LoongArch外设链为:

```

16550 UART (0x1fe001e0)
  → PCH-PIC input 2 (VIRT_UART_IRQ - VIRT_GSI_BASE)
  → PCH-PIC output 0 (htmsi_vector[2] reset=0)
  → EIOINTC line 0
  → EIOINTC pin 0
  → CPU HWI0 = ESTAT.IS[2]

```

hal_irq_init()解除PCH-PIC输入2的mask; 随后通过iocsrdd.w/iocsrwr.w访问IOCSR 0x1400起的EIOINTC寄存器, 使能line 0, 配置IPMAP到pin 0、COREMAP到core 0。hal_irq_hart_init()设置ECFG.LIE[2]。hal_irq_claim()检查ESTAT.IS[2], 读取当前CPU的COREISR并用W1C清位; hal_irq_complete()为空, 因为claim已经完成清除。

UART修复是本阶段最重要的平台调试结论。早期代码认为QEMU 8.2.2不投递UART中断, 因此使用10ms定时器轮询。实际根因有四类:

1. **EIOINTC地址错误**: EIOINTC=0x0fe00000位于QEMU低端RAM; QEMU 8.2.2把EIOINTC放在IOCSR地址空间0x1400~0x1cff。
2. **访问方式错误**: EIOINTC必须用iocsrdd.w/iocsrwr.w访问, 普通volatile指针写入只修改了RAM。
3. **CPU中断位错误**: QEMU virt把EIOINTC pin0接到CPU IRQ2, 对应ECFG.LIE[2]/ESTAT.IS[2], 而不是此前使用的IS[12] (IPI线)。
4. **COREISR的hart偏移错误**: IOCSR访问自带requester_id, 自动选择当前CPU的COREISR bank, 不应再加hart*4。

验证证据如下：

```
QEMU monitor: info mtree
address-space: memory
0000000000000000-00000000ffffffff (ram): loongarch.node0.lowram
0000000010000000-...                : loongarch_pch_pic
000000001fe001e0-...                : serial
address-space: IOCSR
0000000000001400-0000000000001cff  : extioi_iocsr

QEMU -trace（输入一个字符）：
loongarch_pch_pic_irq_handler irq 2 level 1
loongarch_extioi_setirq set extirq 0 level 1
旧驱动的 extioi_readw/writew 次数 = 0
```

第一条证据说明0x0fe00000是RAM而非EIOINTC；第二条说明UART中断已经到达EIOINTC输入，但EIOINTC从未被配置。结合hw/loongarch/virt.c和hw/intc/loongarch_extioi.c源码，才确定IOCSR访问和IS[2]路由。

修正后，hal_uart.c同时打开IER_RX|IER_TX，批量发送走tx_lock+tx_busy+sleep/wakeup+TX-empty IRQ，RX中断排空FIFO并回调consoleintr()；hal_putchar()在普通路径持同一把tx_lock，panic路径绕锁。hal_console_poll()接口被删除，clockintr()不再轮询UART。两架构的hal_irq_claim()统一返回-1表示无中断，因此LA的IRQ 0可以合法参与devintr()分发。

定时器为每CPU CSR设备，未经过EIOINTC。QEMU LoongArch constant timer周期为10ns（100MHz），TIMER_TICK_CYCLES=10000000对应100ms。hal_timer_init()设置ECFG.LIE[11]、TICLR、TCFG（PERIOD|EN）；hal_timer_ack()写TICLR确认当前tick。clockintr()只调用hal_timer_ack()，仅CPU0累加全局ticks，与RISC-V的10Hz语义一致。

7.9 loader-backed三块RAM disk

三个文件系统镜像通过objcopy嵌入内核时，kernel.bin超过QEMU -bios约4MiB的ROM加载上限。当前构建分别装载内核和数据镜像：

内容	装载方式/地址	设备语义
约120KiB纯内核	-bios kernel.bin, 0x1c000000	固件代码
fs.img	QEMU loader, 0x09000000	dev=1，仅xv6fs
ext2.img	QEMU loader, 0x0a000000	dev=2，完整独立ext2
fat32.img	QEMU loader, 0x0b000000	dev=3，独立FAT32

三个窗口各预留16MiB，位于PHYSTOP之上且低于256MiB低端RAM边界，未进入kalloc()管理范围。16MiB为统一驱动容量上限。当前镜像分别约为2MiB、8MiB和10MiB，其中8MiB ext2与当前后端的单块组实现匹配。HAL执行设备号和块边界检查，文件系统后端解释镜像内部格式。

hal_ramdisk.c为每个设备保存基址和独立自旋锁。hal_disk_rw()检查blockno < RAMDISK_STRIDE/BSIZE，在锁内执行一次BSIZE大小的同步memmove()并清除b->disk。驱动未分配镜像页面，也未生成设备中断。写入在当前QEMU进程中可见，退出后未回写宿主镜像。

设备和地址的对应关系由以下计算建立：

```
rd = &ramdisk[dev - 1];
rd->data = RAMDISK_BASE + (dev - 1) * RAMDISK_STRIDE;
offset = b->blockno * BSIZE;
```

Makefile的`check-loongarch-images`在启动QEMU前验证每幅镜像不超过16MiB。

7.10 平台数据通路

7.10.1 控制台输入

LoongArch控制台输入链为：UART触发PCH-PIC line2，经EIOINTC line0到CPU HWIO（IS[2]）；`usertrap()`或`kerneltrap()`调用`devintr()`，`hal_irq_claim()`返回IRQ 0并进入`hal_console_intr()`排空RX FIFO；`consoleintr()`把字符放入输入环形缓冲并唤醒`consoleread()`。整条路径不再依赖定时器轮询。

7.10.2 控制台输出

批量输出由`hal_console_write()`在`tx_lock`内写THR并睡眠等待TX empty中断；`hal_putchar()`为printf/回显提供持锁的同步输出；panic路径绕锁。多CPU同时输出时由`tx_lock`串行化。

7.10.3 dev=2的ext2读取

```
ext2后端 → bread(dev=2, blockno)
→ buffer cache命中：直接返回
→ 未命中：hal_disk_rw(buf, read)
    → ramdisk[1].data + blockno*BSIZE
    → 同步memmove到b->data
→ ext2解析b->data
```

VFS、ext2和buffer cache使用与RISC-V相同的设备号和块接口。平台差异位于`hal_disk_rw()`：RISC-V提交异步virtio请求，LoongArch执行同步RAM复制。

7.11 完成范围与平台约束

当前LoongArch实现覆盖启动、多核同步、每核定时器、用户和内核trap、四级用户页表、软件TLB重填、缺页处理、上下文切换、内核栈guard、EIOINTC/UART RX/TX中断、控制台和三个独立块设备。测试结果记录于第九章。

已知平台约束为RAM disk写入未跨QEMU进程持久化。该约束位于LoongArch块设备实现中，公共控制台、buffer cache和VFS接口保持不变。后续若需持久化，可在同一`hal_disk_*`契约下增加LoongArch PCI virtio后端。

第八章 FAT32文件系统适配

说明：初赛阶段仅支持RISC-V；决赛阶段通过 QEMU loader-backed RAM disk 方案，使 LoongArch 平台也获得 dev=3 的 FAT32 支持，同时新增了 VFAT LFN 长文件名（≤255 字符）完整支持。

8.1 FAT32简介

FAT32（File Allocation Table 32）是微软在FAT16基础上扩展的文件系统，广泛用于U盘、SD卡等可移动存储介质。其特点是结构简单、兼容性好。FAT32的关键数据结构包括：BPB（BIOS Parameter Block，位于扇区0）、FAT表（每个表项32-bit，有效28-bit）、32字节的8.3短目录项。

本实现将FAT32作为xv6 VFS的第三个文件系统后端，通过`vfs_register(fat32_mount, "fat32")`注册，遵循与ext2相同的注册模式。

8.2 文件结构

文件	行数	说明
kernel/fat32.h	91	数据结构：fat32_bpb(BPB), fat32_dirent(32字节目录项), fat32_lfn(13字符LFN条目), FAT常量
kernel/fat32.c	985	VFS后端实现： mount/lookup/read/write/create/unlink/mkdir/readdir/stat/truncate + LFN编解码
user/fat32test.c	311	42项自动化测试：mount/读写/append/truncate/multi-cluster/mkdir/readdir/LFN/unlink/umount
合计	1387	

8.3 适配策略

策略	说明
双平台	RISC-V（virtio磁盘） + LoongArch（QEMU loader-backed RAM disk），均为 dev=3
独立磁盘镜像	FAT32使用第三个块设备(dev=3)，10MB独立镜像
512字节扇区适配	xv6使用BSIZE=1024块，FAT32使用512字节扇区，通过sread()/swrite()辅助函数做扇区→块映射
8.3短名 + VFAT LFN	支持≤255字符长文件名，LFN条目（每13字符一条）与短名条目的完整编解码
多扇区簇	按BPB_SecPerClus 动态计算簇大小，不假设单扇区
双FAT同步更新	每次FAT写入同时更新两份FAT表

8.4 磁盘布局与核心计算

8.4.1 FAT32镜像布局

[扇区 0: BPB + 启动代码 (512B)]	
[扇区 1: FSInfo扇区]	
[扇区 2-31: 保留区]	
[FAT表1: BPB_FATSz32 扇区]	
[FAT表2: BPB_FATSz32 扇区]	
[数据区: 簇2 = 根目录开始]	

8.4.2 核心计算公式

```
FirstDataSector = BPB_RsvdSecCnt + BPB_NumFATs × BPB_FATSz32
FirstSectorOfCluster(N) = ((N - 2) × BPB_SecPerClus) + FirstDataSector
FAT表项偏移 = N × 4 (每项4字节)
FAT表项所在扇区 = BPB_RsvdSecCnt + (偏移 / BPB_BytsPerSec)
```

8.4.3 512字节扇区→1024字节块的转换

这是本次实现中遇到的核心技术难点。xv6的**bread()**/**bwrite()**使用固定块大小**BSIZE=1024**，而FAT32标准使用**BPB_BytsPerSec=512**扇区。每1个xv6块 = 2个FAT32扇区。

```
xv6块0: [扇区0 (512B)][扇区1 (512B)]
xv6块1: [扇区2          ][扇区3          ]
```

所有FAT32磁盘I/O必须通过**sread()/swrite()**辅助函数，封装扇区→块的转换：

```
static int sread(struct fat32_mount *fm, uint sec, uint off, void *buf, uint n) {
    uint blk = sec / fm->spb;           // 扇区号→块号
    uint bo  = (sec % fm->spb) * fm->bps + off; // 块内偏移
    struct buf *bp = bread(fm->dev, blk);
    if (!bp) return -1;
    memmove(buf, bp->data + bo, n);
    brelse(bp);
    return 0;
}
```

其中**spb = BSIZE / bytes_per_sec** (通常1024/512=2)，**bps = BPB_BytsPerSec** (512)。

8.5 数据结构

8.5.1 BPB (BIOS Parameter Block)

位于FAT32镜像的第0扇区。关键字段：

字段	偏移	大小	说明
BPB_BytsPerSec	11	2	每扇区字节数，必须为512
BPB_SecPerClus	13	1	每簇扇区数
BPB_RsvdSecCnt	14	2	保留扇区数，FAT32典型值32
BPB_NumFATs	16	1	FAT表份数，通常为2
BPB_TotSec16/32	19/32	2/4	总扇区数 (≤32MB时TotSec16有效)
BPB_FATSz32	36	4	FAT32特有：每个FAT表的扇区数

字段	偏移	大小	说明
BPB_RootClus	44	4	根目录起始簇号，通常为2

8.5.2 目录项 (32字节)

Offset	Size	Field
0	11	DIR_Name (8.3短文件名，空格填充，首字节0xE5=已删除)
11	1	DIR_Attr (ATTR_DIRECTORY=0x10, ATTR_ARCHIVE=0x20)
20	2	DIR_FstClusHI (起始簇号高16位)
26	2	DIR_FstClusLO (起始簇号低16位)
28	4	DIR_FileSize (文件大小, uint32)

8.5.3 FAT表项

FAT32每个表项32-bit（有效28-bit，高4位保留）：

值	含义
0x00000000	空闲簇
0x0FFFFFFF	坏簇
>= 0x0FFFFFFF	簇链结束(EOC)

8.6 已实现的VFS操作

vnode_ops	实现函数	实现要点
lookup	flookup	扫描目录项(32字节为单位)，LFN重组 → 完整长名匹配；短名按8.3名匹配（大写转换+空格padding）
read	fread	遍历FAT簇链，通过sread()逐扇区读取数据，支持任意簇大小
write	fwrite	写入数据，必要时alloc_clus扩展簇链，通过swrite()写回；写后同步目录项大小（B1）
create	fcreate	判断是否需LFN（fits_83）→ gen_short_name生成唯一短名 → dwrite_lfn写LFN链+短名项
unlink	funlink	标记目录项DIR_Name[0]=0xE5 + count_lfn_backwards连LFN项一并标记删除 + free_chain释放FAT簇链
mkdir	fmkdir	委托fcreate(V_DIR)，全簇清零写入"././.", 多扇区簇完整初始化
readdir	freaddir	向后扫描重组LFN，返回完整长名（≤255，经vdirent / VDIRSIZ=256）
stat	fstat	返回文件类型(V_FILE/V_DIR)和大小（从目录项维护）
truncate	ftrunc	free_chain释放簇链+重置file_size=0+同步目录项

核心helper函数：

- `clus_to_sec(N)` — 簇号→数据区首个扇区号
- `read_fat(N) / write_fat(N, val)` — 读写FAT表项（含双FAT同步更新）
- `alloc_clus() / free_chain(start)` — 簇分配/释放
- `lfn_checksum(shortname) / dwrite_lfn / dlookup` — LFN 校验和、写链、查链
- `sync_dirent(vp)` — 将内存中的文件大小/首簇回写目录项（保证重开后可见）

8.7 LoongArch 支持说明

决赛阶段已成功支持 LoongArch FAT32。

方案：LoongArch QEMU virt 通过 `-device loader,file=fat32.img,addr=0x0b000000,force-raw=on` 将 FAT32 镜像作为独立 RAM disk 加载（dev=3），与 RISC-V 的第三个 virtio 磁盘（`-drive file=fat32.img`）保持一致的设备号语义：

```
RISC-V:      -drive file=fat32.img,id=x2 → virtio-mmio-bus.2 → dev=3
LoongArch:  -device loader,file=fat32.img,addr=0x0b000000 → RAM disk → dev=3
```

两个平台的内存布局统一：`fs.img=0x09000000`、`ext2.img=0x0a000000`、`fat32.img=0x0b000000`（每镜像 16MiB 窗口），HAL 只做越界检查，不理解文件系统格式。这使三文件系统在双平台上行为完全一致。

8.8 测试验证

8.8.1 测试环境

在 RISC-V / LoongArch Docker 容器中分别执行：

```
cd /xv6/xv6-riscv-xv6-riscv-rev5
make clean && make qemu          # RISC-V
# 或
make ARCH=loongarch clean && make ARCH=loongarch qemu  # LoongArch
```

启动后在 xv6 shell 中输入 `fat32test` 运行自动化测试（双平台通用）。

8.8.2 测试结果

```
$ fat32test
fat32test: starting
PASS: mount /fat 3 fat32
PASS: rwcheck content /fat/hello.txt (16 bytes)
PASS: rwcheck size /fat/hello.txt (16)
PASS: hello.txt type T_FILE
PASS: O_APPEND content
PASS: O_TRUNC size zero
PASS: write after truncate yields size == written bytes (FAT32)
PASS: rwcheck content /fat/multi.bin (10000 bytes)
PASS: mkdir /fat/subdir
PASS: subdir contains .
PASS: subdir contains ..
```

```
PASS: lookup /fat/subdir/.
PASS: lookup /fat/subdir/..
PASS: lookup /fat/..
PASS: root has hello.txt
PASS: readdir shows LFN <=13 verbatim (case preserved)
PASS: readdir shows full LFN >13 name
PASS: reopen LFN >13 by full name
PASS: unlink LFN file
PASS: unlink non-empty dir rejected
PASS: umount /fat
fat32test: ALL PASSED
```

42项测试全部通过，覆盖：挂载/卸载(dev=3)、读写往返+大小持久化、O_APPEND、O_TRUNC、多簇大文件(10KB)、mkdir+嵌套、子目录readdir、"/路径查找、根目录readdir、LFN≤13大小写保留、LFN>13完整长名显示+按全名重开、unlink（普通/LFN/非空目录拒绝）、umount。

8.9 mount失败的三个关键问题与解决方案

问题1: 512字节扇区 vs 1024字节块

现象：mount /fat 3 fat32失败，BPB解析不正确。

根因：xv6的bread()/bwrite()使用BSIZE=1024固定块大小。直接用FAT32扇区号调用bread()会导致位置错误——扇区0和扇区1被当作同一个1024字节块读取。

解决方案：引入sread()/swrite()辅助函数，自动完成blkno=sec/spb; offset=(sec%spb)*bps的转换。

问题2: BPB_TotSec16不为0

现象：以BPB_TotSec16 != 0作为非FAT32的判断条件，导致小镜像mount被拒绝。

根因：FAT32规范要求TotSec16=0（使用TotSec32），但Linux的mkfs.fat对≤32MB的小镜像将总扇区数填入TotSec16（16位够用），TotSec32保持为0。

解决方案：使用TotSec16 ? TotSec16 : TotSec32作为总扇区数，FAT32类型判定仅依赖BPB_FATSz32 != 0。

问题3: xv6fs inode耗尽

现象：mkdir /fat在shell中手动执行失败。

根因：xv6fs使用固定大小的inode表(13 blocks × 16 inodes/block = 208 inodes)，用户程序消耗大量inode。

解决方案：在init.c启动时预先创建/fat目录（与/mnt相同方式处理）。

8.10 已知限制

限制	说明
LFN仅ASCII	长文件名仅支持 ASCII 字符 (<128)；非 ASCII 的 UTF-16 条目被跳过
无时间戳	FAT32 目录项含时间字段，但本实现不主动维护 atime/mtime/ctime
无子目录深度限制检查	理论可创建任意深度目录

限制	说明
10MB镜像上限	mkfs.fat对10MB以下镜像有警告(不影响使用)
无并发保护	多核并发FAT I/O未加锁（xv6测试场景为单核/单线程shell）

第九章 测试与验证

9.1 测试环境

项目	版本/配置
QEMU (RISC-V)	7.2.22
QEMU (LoongArch)	8.2.2
编译器	riscv64-linux-gnu-gcc 12.2.0 / loongarch64-linux-gnu-gcc 14.2.0
操作系统	Debian bookworm (RISC-V容器) / Ubuntu 24.04 (LoongArch容器)

9.2 xv6原生测例（决赛版）

测试	RISC-V	LoongArch
usertests -q	✅ ALL PASSED	✅ ALL PASSED
forktest	✅	✅
grind	✅	✅

9.3 赛方测试用例（决赛版）

测试	来源	RISC-V 结果	LoongArch 结果
test1（文件拷贝×4组合）	赛方 test1.c（\$U/ 下）	✅ 全部 succeed	✅ 全部 succeed
test2（跨FS拷贝+内容验证）	赛方 test2.c（\$U/ 下）	✅ succeed	✅ succeed

9.4 自定义综合测试（ext2test，L1~L4，决赛版）

当前结果（决赛版）：15/15 全部 PASS（RISC-V 与 LoongArch 均通过）

层次	测试项	结果
L1.1	mount→umount→remount 循环	PASS
L1.2	mount 错误路径	PASS
L2.1	create + lookup 嵌套	PASS
L2.2	跨块读写 4096 字节	PASS
L2.3	readdir 验证	PASS
L2.4	mkdir + type 检查	PASS

层次	测试项	结果
L2.5	unlink 语义	PASS
L2.6	硬链接 nlink 验证	PASS
L2.7	mknod 设备节点	PASS
L2.8	stat 完整性	PASS
L2.9	truncate (O_TRUNC)	PASS
L3	namei 路径遍历 (5 子项)	PASS
L4.1	并发 write (fork)	PASS
L4.2	unlink-while-open	PASS
L4.3	大文件 200 块 (间接块)	PASS

初赛阶段 L4.1（并发write）与 L4.3（大文件间接块）曾失败，决赛版已修复：L4.1 修复了 ext2 锁粒度与 begin_op/end_op 包裹范围；L4.3 修复了间接块分配/清零路径。

9.5 赛题参考测试结果

运行方式（RISC-V 与 LoongArch 通用）：

```
$ cat demo.sh | sh      # 自动挂载 + 运行全部测试
# 或手动执行：
$ mount /mnt 2 ext2     # 挂载 ext2 到 /mnt
$ test1                 # 赛方测试1：跨文件系统拷贝
$ test2                 # 赛方测试2：跨FS拷贝+内容验证
$ ext2test               # 自研综合测试（L1~L4 共15项）
```

执行 test1 与 test2 的输出参考：

```
----- test1: begin... -----{
test_copy_file(/mnt/hello.c,/hello.c): succeed
test_copy_file(/mnt/hello.c,/tests/hello.c): succeed
test_copy_file(/hello.c,/mnt/hello.c): succeed
test_copy_file(/tests/hello.c,/mnt/hello.c): succeed
----- test1: finished -----}

----- test2: begin... -----{
将从 xv6 文件系统复制文件到 EXT2 文件系统
文件复制成功!
验证复制结果:
复制成功! 文件内容:
#include "kernel/types.h"
#include "user/user.h"
void main () { printf ("Hello, world!\n"); }
test_cross_fs_copy(): succeed
----- test2: finished -----}
```

9.6 FAT32测试（决赛版）

测试程序： `user/fat32test.c`（311行），42项自动化测试

双平台测试结果：RISC-V 与 LoongArch 均为 **42/42 全部 PASS**

测试类别	覆盖内容	结果
挂载/卸载	mount("/fat",3,"fat32") / umount	PASS
读写往返	16B/20B/6B 内容+大小持久化（重开后 size 正确）	PASS
O_APPEND	追加写入 "abc"+"def" → "abcdef"	PASS
O_TRUNC	截断后 size=0；重写后 size==写入字节数（FAT32语义）	PASS
多簇大文件	10KB 跨多簇读写往返	PASS
mkdir + 嵌套	子目录创建 + 嵌套文件 + 子目录 readdir(/..)	PASS
"./".." 查找	<code>/fat/subdir/..</code> 、 <code>/fat/subdir/...</code> 、 <code>/fat/..</code>	PASS
根目录 readdir	hello/append/trunc/multi/subdir 全部可见	PASS
LFN ≤13	大小写保留显示（FileDataX.txt）	PASS
LFN >13	完整长名显示 + 按全名重开成功 （LongFileNameTest.txt）	PASS
unlink	普通/LFN 删除 + 非空目录拒绝 + 空目录删除	PASS

9.7 HAL 专项验证（决赛最新提交）

验证项	配置/命令	结果
RISC-V 完整 usertests	<code>usertests -q</code>	ALL PASSED

验证项	配置/命令	结果
LoongArch 3 CPU 完整 usertests	make ARCH=loongarch CPUS=3 qemu 后 usertests -q	ALL PASSED
LoongArch 抢占	usertests preempt	通过
LoongArch UART 交互输入	串口输入回显、shell 命令执行	通过，无定时轮询
RISC-V HAL_NDISK=1	编译	通过
RISC-V HAL_NDISK=2	编译	通过
默认 HAL_NDISK=3	双架构编译与运行	通过
git diff --check	最新提交	通过

该组验证重点覆盖 AP 定时器、trampoline 映射失败路径、LA UART RX/TX 中断、定时器 10Hz 语义一致性，以及删除低两页后的MAXVApplus和双架构usertests -q回归。

9.8 从初赛到决赛的已解决问题

问题	初赛状态	决赛修复
L4.1 并发 write FAIL	ext2 写入交错	修复锁粒度与 begin_op/end_op 包裹
L4.3 大文件间接块 FAIL	间接块分配/清零 bug	修复 bmap alloc=1 间接块路径
LoongArch 无 FAT32	QEMU ROM 4MB 限制无法塞入镜像	loader-backed RAM disk （addr=0x0b000000）独立加载，不再受 ROM 限制
FAT32 readdir 仅 13 字符	长文件名被截断、只能显示短名	VDIRSIZ=256 + dirent 统一 ABI，LFN 完整显示
长文件名 unlink 崩溃	内核 panic: kerneltrap (scause=0xf)	sysfile.cchar name[14] 栈溢出 → name[VDIRSIZ]
fat32test 用户态崩溃	目录列表 12KB 放栈上溢出 guard 页	移入堆（malloc）
LA 收不到 UART 中断	误判为 QEMU 8.2.2 不支持，使用 10ms 轮询	EIOINTC 改经 IOCSR 0x1400 访问；HWI 从 IS[12] 改为 IS[2]；恢复 RX/TX 中断驱动
LA AP 无定时器	AP 空闲后可能无法唤醒，用户进程无抢占	main() AP 路径调用 hal_timer_init()
trampoline 映射失败被忽略	OOM 时可能产生残缺用户页表	proc_pagetable() 检查返回值并释放页表
LA 低两页保留缺少架构依据	历史防御增加专用接口和每进程物理页	删除低页映射及生命周期接口；保留 RVACFG/MAXVA/refill上界检查

问题	初赛状态	决赛修复
双平台 tick 周期不一致	LA 注释误认为 10ms，实际 100ms	统一 <code>hal_timer_init/hal_timer_ack</code> ，双平台均为 10Hz
汇编/C 布局魔法数漂移	修改结构体或布局后可能静默错位	trapframe/context 偏移断言、KSTACK/guard 宏、链接期 refill 页断言

第十章 设计精巧度分析

10.1 与Linux VFS对比

维度	Linux	本作品
VFS代码量	> 10万行	669行 (vfs.c + vfs.h)
支持FS类型	数十种	3种 (xv6fs + ext2 + FAT32)
核心抽象	多层继承 (dentry/inode/address_space/...)	统一vnode+ops
设计目标	高性能通用	教学精简

10.2 整体代码量统计

类别	文件	行数
VFS核心	kernel/vfs.c + kernel/vfs.h	557 + 112 = 669行
ext2	kernel/ext2.c + kernel/ext2.h	1028 + 128 = 1156行
xv6fs	kernel/xv6fs.c	497行
上层适配	kernel/sysfile.c + kernel/file.c	386 + 163 = 549行
FAT32	fat32.c/h	1076行
HAL	hal/ (RISC-V + LoongArch + 接口层, 含链接脚本)	3485行 (接口234 + RISC-V 1608 + LoongArch 1643)
统计口径说明	本表为当前文件规模；HAL含从原xv6迁入的RISC-V平台代码，不能把3485行全部视为净新增。净新增主要集中在 LoongArch 平台、接口层和 RISC-V 多盘扩展	

10.3 设计模式

模式	应用位置	效果
策略模式	vnode_ops	每FS独立实现，VFS层不感知
注册表模式	fstypes[]	添加新FS仅需一行注册
句柄模式	vnode	所有文件操作统一入口

模式	应用位置	效果
编译期策略绑定	HAL 的 ARCH/HAL_NDISK	一份内核镜像绑定一个平台和设备数，无运行时间接分派
生命周期契约	hal_vm_map/unmap_trampoline	RISC-V特殊映射要么完整建立，要么失败回滚；LoongArch维持同名空操作
编译期 ABI 断言	_Static_assert、链接脚本 ASSERT	C 结构体、汇编偏移和链接布局互相锁死

10.4 与参考项目对比

维度	本作品	静春山（SpringOS）	RuOK
VFS代码行数	669行	~200行（薄适配层）	未统计
ext2/4代码行数	1156行（ext2手写）	~4万行（集成lwext4库）	未统计
支持FS数量	3	1（仅ext4）	未统计
独立HAL	是	是	未统计
设计路线	手写教学实现	集成成熟库	未统计

第十一章 挑战与展望

11.1 技术难点回顾

- ext2 orphaned延迟释放的并发安全
- "."跨文件系统返回
- mount失败路径的完整资源清理
- VFS路径穿越的性能优化
- LoongArch软件TLB重填中的地址范围、偶/奇页和无效项处理
- DMW0与页表并存时实现可检测的内核栈guard
- 在QEMU ROM和外设模型约束下保持dev=1/2/3三块独立设备语义
- QEMU LoongArch UART中断链的定位：EIOINTC属于IOCSR空间而非MMIO，CPU中断位为IS[2]而非IS[12]
- 多核定时器语义统一：AP定时器、10Hz tick、hal_timer_ack抽象
- 用编译期和链接期断言消除汇编偏移、栈窗口和链接布局中的魔法数

11.2 未来工作

- 可选实现LoongArch PCI virtio磁盘驱动，用于需要镜像持久化的环境；当前教学Demo继续保留更小、更稳定的loader RAM disk
- 在真实LoongArch开发板上复核EIOINTC IOCSR访问和UART中断链，验证HAL不局限于QEMU
- 继续收敛kernel/trap.c中的中断号分发，将其抽象为HAL设备类型或平台dispatch
- FAT32 长文件名(LFN)的 UTF-16 非 ASCII 支持（当前仅 ASCII）
- FAT32 时间戳（atime/mtime/ctime）维护
- 更多文件系统（tmpfs、procfs）
- VFS写缓存与预读
- FAT32/FAT 表并发安全保护（为 fm->lock 真正加锁）
- ext2 双重/三重间接块、多块组支持

11.3 收获与体会

本项目通过从零设计并实现硬件抽象层(HAL)和虚拟文件系统(VFS)，在真实的xv6教学操作系统上实践了操作系统的两大核心抽象——跨平台硬件虚拟化和多文件系统统一访问。主要收获：

- 架构设计的价值：**HAL的三明治架构和VFS的vnode_ops模式证明了"先设计接口，再分别实现"是控制复杂度的有效方法。添加FAT32仅需一行`vfs_register`，正是这种设计的直接收益。
- 512字节扇区转换：**xv6固定使用1024字节块，FAT32使用512字节扇区。`sread()/swrite()`通过块号和块内偏移完成转换，文件系统代码未假设块大小等于扇区大小。
- LoongArch块设备方案演进：**初赛阶段受QEMU ROM 4MiB上限影响，嵌入式镜像方案仅支持xv6fs和ext2。决赛阶段使用QEMU `-device loader`把三幅独立镜像加载到固定RAM地址，fs/ext2/fat32分别映射到dev=1/2/3。该方案通过调整装载模型满足容量要求，并保持RAM disk驱动规模稳定。
- AI辅助开发的边界：**AI工具用于代码框架生成、代码审查和文档整理。512字节扇区映射、ROM容量、PTE权限和UART寄存器行为由开发者结合源码、手册和运行结果复核。决赛后期，AI通过QEMU `info mtree`、trace事件和QEMU 8.2.2源码定位了LA UART中断链的四个根因（EIOINTC地址空间、IOCSR访问指令、CPU中断位、COREISR per-hart视图），并给出AP定时器、失败路径、布局断言、接口清理等优化建议；最终方案由开发者复核后拆分提交。
- 从“能跑”到“可证明”：**HAL不是一次性搬移平台代码。通过统一`hal_irq_claim`返回值、`hal_timer_init/ack`、UART `tx_lock`协议，并用`_Static_assert`锁死trapframe/context/栈窗口/refill页，很多潜在错误在编译期或链接期暴露，而不是等到多核压测或现场演示。

附录

A. 编译与运行

```
# 编译
make

# 制作ext2和FAT32镜像
make ext2.img
make fat32.img

# 运行
make qemu

# ext2测试（dev=2，挂载点 /mnt）
mount /mnt 2 ext2
test1          # 赛方：跨FS文件拷贝
test2          # 赛方：跨FS拷贝+内容验证
ext2test       # 自研 L1~L4 综合测试（15 项）

# FAT32测试（dev=3，挂载点 /fat，由 init.c 自动创建并挂载）
fat32test      # 42 项综合测试
```

B. O_APPEND 与 O_TRUNC 支持

O_APPEND 与 O_TRUNC 在决赛版中已完整实现并于用户态测试验证：

- **O_TRUNC**: `sys_open` 中检测到 `O_TRUNC` 且目标为普通文件时调用 `vfs_truncate(vp)` → 具体 FS 的 `ops->truncate` 释放全部数据块并清零大小。
- **O_APPEND**: `sys_open` 中设置 `f->appendable=1`，并把初始偏移置为 `vp->size`（文件末尾）。之后 `filewrite` 写入前若为 `appendable` 则自动从文件尾开始。
- **测试验证**: `fat32test` 中 `O_APPEND`（"abc"+"def"→"abcdef"）与 `O_TRUNC`（截断后 `size=0`、重写后 `size==`写入字节数）均 PASS；`ext2test L2.9 truncate` 亦 PASS。
- **FAT32语义说明**: `FAT32`文件大小存储于目录项，`ftruncate`将`size=0`持久化回磁盘。`O_WRONLY`（无`TRUNC`）重写时采用当前`FAT32`后端语义，`size`等于实际写入字节数；`POSIX`覆盖写通常保留未覆盖的原文件尾部。

C. 参考资料

- `xv6-riscv-rev5`: <https://github.com/mit-pdos/xv6-riscv>
- 赛题测试仓库: <https://github.com/yanjun-wen/xv6-extend-vfs>
- 参考项目1(静春山/SpringOS): <https://gitlab.eduxiji.net/educg-group-36002-2710490/T202510558995330-264>
- 参考项目2(RuOK): <https://gitlab.eduxiji.net/educg-group-36002-2710490/T202510486995232-2402>

D. 使用AI协助开发

本项目在开发过程中使用了AI辅助编程工具（Claude Code框架 + DeepSeek V4 API）。

项目	说明
AI框架	Claude Code (Anthropic官方CLI工具)
底层模型	DeepSeek V4 API
使用模式	交互式对话进行方案讨论、代码审查、项目文档与代码的一致性检查、编译调试、测试方案设计
工作目录	<code>xv6-riscv-xv6-riscv-rev5/</code>

D.1 使用方式：AI提供审查和候选方案，开发者负责决策闭环

HAL开发通过跨阶段背景记录文档保存接口清单和问题记录，并通过阶段性代码审查记录特定代码快照的风险清单。AI输出作为候选方案进入以下复核流程：

AI扫描代码并提出风险/候选方案

→ 开发者定位到当前源码和体系结构手册

→ 构造最小复现或检查QEMU寄存器行为

→ 比较代码量、稳定性和HAL边界

→ 开发者选择、修改或否决方案

→ 单独提交并执行双架构编译/运行测试

→ 更新设计文档和已知限制

AI主要提供代码审查、项目文档与代码的一致性检查、跨架构条件块和调用关系的整理等辅助工作。开发者负责硬件语义核验、最终架构决策、代码修改、测试结果解释和提交拆分。

D.2 阶段性审查建议的采纳过程

阶段性代码审查中指出：通用层仍有10个LoongArch条件块、内核栈guard被DMW0绕过、存在两套TLB填充、UART固定延迟不可靠，以及双包装和死代码。开发者结合当前源码、体系结构约束和测试结果逐项复核，处理结果如下。

审查项	AI建议	复核结果与最终实现	证据
HAL边界泄漏	把10个条件块下沉	按资源生命周期合并成7项HAL职责， <code>kernel/</code> 架构条件块归零	<code>f08c90f</code> 、 <code>f731aed</code>
LoongArch guard失效	可接受物理隔离并跳过无效映射，或重新设计	实现高地址栈、固定PGDH、无PTE guard、refill范围检查和紧急诊断栈	<code>63bed2d</code>
<code>tlb_fill_from_pte()</code>	删除约30行重复路径	确认偶/奇半项别名和全量TLB刷新行为后删除； <code>vmfault()</code> 建立PTE，汇编refill装入TLB	<code>0b463d3</code>
双包装和死代码	删除旧PLIC/UART/swtch名字和未调用接口	删除转发接口； <code>hal_console_intr(handler)</code> 通过回调连接通用console层	<code>7bb6704</code>
LoongArch低地址内核页表	DMW0下这些映射不会被查询	<code>hal_vm_map_kernel()</code> 改为空实现； <code>proc_mapstacks()</code> 继续建立PGDH使用的高地址栈映射	<code>91cfd94</code>
UART固定延迟	重新验证LSR或合并驱动	先改用LSR bit 5；后续确认EIOINTC应通过IOCSR访问，修正HWI路由并统一RX/TX中断状态机	<code>66acbd8</code> 及后续UART修正
PTE权限	审查未完整覆盖NR/NX语义	开发者进一步发现数据页默认可执行，引入集中权限翻译，只在通用 <code>mappages()</code> 增加一次调用	<code>c5cfd4b</code>
RAM disk命名和布局	建议设备接口统一、文件名归真	完成接口和命名清理；装载方案使用QEMU loader三独立窗口，镜像未嵌入内核	<code>c7c4a1c</code> 、 <code>abdbb9</code> 、 <code>91cfd94</code>
LA AP 无定时器	每个hart进入调度器前初始化本地定时器	<code>main()</code> AP 路径调用 <code>hal_timer_init()</code> ，修复 idle 唤醒和抢占	<code>7c349d3</code>
trampoline 映射失败被忽略	检查返回值并在失败时释放页表	<code>proc_pagetable()</code> 失败即 <code>uvmfree</code> 并返回0	<code>7c349d3</code>
低两页保留机制	抽象平台最小进程大小	初期曾增加生命周期接口；后续复核ISA/ABI并实测后确认为冗余，最终整体删除	<code>7c349d3</code> 、当前修订

审查项	AI建议	复核结果与最终实现	证据
UART 同步输出只关本核中断	同步/批量/中断路径共用 <code>tx_lock</code>	两平台 <code>hal_putchar()</code> 持 <code>tx_lock</code> 并置 <code>tx_busy</code> ; panic 绕锁	7c349d3
定时器接口暴露绝对 deadline, 两平台 tick 频率不一致	改为 <code>hal_timer_init/hal_timer_ack</code> , 平台自藏 timebase	RISC-V 10MHz×1e6、LA 100MHz×1e7, 均为 10Hz; 仅 CPU0 累加 ticks	a7f679c
汇编中复制布局魔法数	汇编直接引用 <code>memlayout.h</code> 宏	<code>KSTACK_*</code> 、 <code>HAL_VA_BITS</code> 单一 定义; C/链接期断言	a7f679c
C 结构体与汇编偏移无 校验	增加 <code>_Static_assert</code>	<code>trapframe</code> 全部偏移和大小、 双架构 <code>hal_context</code> 偏移和大 小断言	a7f679c
磁盘接口无独立头文 件、 <code>VIRTIO_NDISK</code> 泄 漏到通用层	新增 <code>hal_disk.h</code> , 统一 <code>HAL_NDISK</code>	<code>bio.c/Makefile/平台驱动</code> 统 一使用 <code>HAL_NDISK</code> ; <code>virtio.h</code> 移入 <code>hal/riscv/</code>	a7f679c

D.3 AI输出的复核范围

1. AI建议对应特定代码快照。 阶段性审查描述了当时存在的10个条件块和失效guard。后续PGDH高地址栈映射改变了“LA页表未使用”的适用范围，因此每项建议均基于最新调用关系和diff重新核对。
2. PTE位编码依据手册和源码校验。 早期说明曾把LoongArch PTE写成“PPN分两段”，也曾直接套用RISC-V正权限。当前实现以LA64手册和`arch.h`为依据：物理地址保持页对齐字段，NR/NX由`hal_pte_encode_perm()`翻译。AI用于整理差异表，位级结论由手册、源码和测试确认。
3. 模拟器行为通过内存树、trace、源码和端到端测试共同确认。 QEMU `info mtree` 显示 EIOINTC 位于 IOCSR `0x1400~0x1cfff`，而旧地址 `0x0fe00000` 是低端 RAM；`-trace` 显示 UART 输入可到达 `pch_pic irq 2` 和 `extioi_setirq irq 0`，但旧代码没有产生任何 `extioi_readw/writew`。结合 QEMU 8.2.2 源码确认：EIOINTC 需用 `iocsrrd.w/iocsrwr.w` 访问，virt 机把 EIOINTC pin0 接到 CPU IRQ2（`ESTAT.IS[2]`），且 COREISR 通过 IOCSR `requester_id` 自动选择当前 CPU。修正后 RX/TX 中断可用，两架构 `usertests -q` 均通过。
4. TLB refill具有独立执行环境。 入口处于`DA=1/PG=0`，普通栈和寄存器保存条件不成立；KSave0/1由用户trap路径占用。汇编实现保存临时寄存器、设置4KiB页大小、检查PGD各级V位，并在失败时填V=0条目转入普通缺页异常。验证范围包括启动、缺页和用户测试。
5. 代码量与风险联合评估。 删除重复TLB填充、包装层、死映射和无架构依据的低两页机制，同时减少代码和风险。内核栈guard增加了必要的汇编检查和异常诊断代码，并把静默内存破坏转换为确定panic。各项修改按照运行风险、接口边界和代码规模共同评估。

D.4 人工贡献与可复核性

HAL的核心设计选择由开发者完成，包括：保持`MAXVA=1<<38`、使用编译期HAL、使用loader完成三镜像装载、把三个文件系统设置为独立设备、为DMW0配置PGDH guard、集中翻译NR/NX权限，以及最终否决“QEMU不支持UART中断”的旧结论、恢复EIOINTC/IOCSR中断驱动收发。

为了使AI辅助过程可复核，每一类优化尽量拆成独立Git提交，提交信息记录问题、替代方案、改动文件和原因；最终文档以当前源码、构建结果和测试记录为准，而不以对话文本或AI建议本身作为完成证据。

D.5 UART BUG 的定位与修复过程（AI 参与）

1. 现象：初版 LA 控制台依赖 `clockintr()` 每 10ms 轮询 `hal_console_poll()`，并注释“QEMU 8.2.2 不投递 UART 中断”。AI 复查时发现该结论与 QEMU 16550 模型能力不符，建议重新验证中断链。
2. 内存树定位：QEMU monitor `info mtree` 显示：
 - 普通内存空间只有 `0x1fe001e0 serial`、`0x10000000 loongarch_pch_pic`；
 - `address-space: IOCSR` 中才是 `0x1400~0x1cff extioi_iocsr`；
 - 旧代码使用的 `0x0fe00000` 落在 `loongarch.node0.lowram` 里，因此所有 EIOINTC 初始化只是改写 RAM。
3. trace 验证：输入字符时可见 `loongarch_pch_pic_irq_handler irq 2 level 1` 与 `loongarch_extioi_setirq extirq 0 level 1`，但 `loongarch_extioi_readw/writew` 计数为 0。说明 UART→PCH-PIC→EIOINTC input 已通，EIOINTC 自身从未被配置。
4. QEMU 源码核对：`hw/loongarch/virt.c` 把 serial IRQ 接到 PCH-PIC input2，PCH-PIC output0 接到 EIOINTC line0，EIOINTC pin0 接到 CPU IRQ2；`hw/intc/loongarch_extioi.c` 的寄存器由 IOCSR 指令访问并按 `requester_id` 选择 CPU。
5. 最终修复：
 - `memlayout.h`：EIOINTC_IOCSR_BASE=0x1400；
 - `arch.h`：新增 `iocsr_read_w/iocsr_write_w`，`ECFG_LIE_HWI/ESTAT_IS_HWI` 改为 `1<<2`；
 - `hal_intr.c`：ENABLE/IPMAP/COREMAP 按 packed 字段读写，COREISR 不再加 `hart*4`，claim 清位；
 - `hal_uart.c`：同时打开 `IER_RX|IER_TX`，采用 `tx_lock+tx_busy+sleep/wakeup`；
 - `kernel/trap.c`：删除 `hal_console_poll()` 轮询路径，IRQ 0 合法化，两架构 `hal_irq_claim()` 统一返回 -1。
6. 回归验证：关掉轮询后，串口输入回显和 `usertests -q` 通过；修复拆分为 `f94c185`，后续在 `7c349d3` 补齐跨 CPU UART 锁。

D.6 AI 给出的 HAL 优化建议及采纳情况

除 UART 定位外，AI 还给出以下优化建议，开发者逐项复核，并按最终依据采纳或撤销：

AI 建议	复核结论	落地提交
所有 hart 都应初始化本地定时器	采纳；AP 否则无法从 <code>wfi</code> 唤醒，且无抢占	7c349d3
检查 <code>hal_vm_map_trampoline()</code> 返回值	采纳；OOM 时不得产生残缺页表	7c349d3
低两页保留建议	初期采纳；后续核对LoongArch规范并在QEMU 8.2.2删除低两页后完成双架构回归，最终撤销该接口	7c349d3、当前修订
UART 同步/批量/TX 中断共用 <code>tx_lock</code>	采纳； <code>hal_putchar()</code> 也置 <code>tx_busy</code> ，panic 绕锁	7c349d3
双平台 tick 频率统一为 10Hz，接口去掉绝对 deadline	采纳；改为 <code>hal_timer_init/hal_timer_ack</code>	a7f679c
汇编引用 <code>memlayout</code> 宏，增加编译/链接期断言	采纳；消除 <code>-193/-1/38</code> 等魔法数	a7f679c

AI 建议	复核结论	落地提交
trapframe/context 偏移断言	采纳；C 与汇编 ABI 编译期锁死	a7f679c
新增 hal_disk.h、HAL_NDISK，移动 virtio.h，删除空 hal_memlayout.h	采纳；通用层不再出现 virtio 命名	a7f679c
统一 hal_irq_claim() 的“无中断”返回值	采纳；RISC-V 的 PLIC 0 转为 -1，LA IRQ0 合法	f94c185、a7f679c

D.7 FAT32 与测试程序

FAT32 文件系统及 fat32test 测试程序的开发过程中，AI 工具参与了测试方案的设计与测试用例的组织。512 字节扇区与 1024 字节 buffer 块的映射问题由开发者在 RISC-V 挂载测试中定位并修复（详见第八章 8.9 节）；LoongArch 平台的 FAT32 支持采用 QEMU loader-backed RAM disk 方案完成，设备语义与 RISC-V 保持一致。